

Context switches, which are essential in a multiprogrammed environment, introduce several operating system design challenges. For one, context switches must be essentially transparent to processes, meaning that the processes are unaware they have been removed from the processor. During a context switch a processor cannot perform any "useful" computation—i.e., it performs tasks that are essential to operating systems but does not execute instructions on behalf of any given process. Context switching is pure overhead and occurs so frequently that operating systems must minimize context-switching time.

The operating system accesses PCBs often. As a result, many processors contain a hardware register that points to the PCB of the currently executing process to facilitate context switching. When the operating system initiates a context switch, the processor safely stores the currently executing process's execution context in the PCB. This prevents the operating system (or other processes) from overwriting the process's register values. Processors further simplify and speed context switching by providing instructions that save and restore a process's execution context to and from its PCB, respectively.

In the IA-32 architecture, the operating system dispatches a new process by specifying the location of its PCB in memory. The processor then performs a context switch by saving the execution context of the previously running process. The IA-32 architecture does not provide instructions to save and restore a process's execution context, because the processor performs these operations without software intervention.²¹

Self Review

1. From where does an operating system load the execution context for the process to be dispatched during a context switch?
2. Why should an operating system minimize the time required to perform a context switch?

Ans: 1) The process to be dispatched has its context information stored in its PCB. 2) During a context switch, a processor cannot perform instructions on behalf of processes, which can reduce throughput.

3.4 Interrupts

As discussed in Chapter 2, Hardware and Software Concepts, interrupts enable software to respond to signals from hardware. The operating system may specify a set of instructions, called an **interrupt handler**, to be executed in response to each type of interrupt. This allows the operating system to gain control of the processor to manage system resources.

A processor may generate an interrupt as a result of executing a process's instructions (in which case it is often called a **trap** and is said to be **synchronous** with the operation of the process). For example, synchronous interrupts occur when a process attempts to perform an illegal action, such as dividing by zero or referencing a protected memory location.

Interrupts may also be caused by some event that is unrelated to a process's current instruction (in which case they are said to be **asynchronous** with process execution; see the Operating Systems Thinking feature, Asynchronism vs. Synchronism). Hardware devices issue asynchronous interrupts to communicate a status change to the processor. For example, the keyboard generates an interrupt when a user presses a key; the mouse generates an interrupt when it moves or when one of its buttons is pressed.

Interrupts provide a low-overhead means of gaining the attention of a processor. An alternative to interrupts is for a processor to repeatedly request the status of each device. This approach, called **polling**, increases overhead as the complexity of the computer system increases. Interrupts eliminate the need for a processor to repeatedly poll devices.

A simple example of the difference between polling and interrupts can be seen in microwave ovens. A chef may either set a timer to expire after an appropriate number of minutes (the timer sounding after this interval interrupts the chef), or the chef may regularly peek through the oven's glass door and watch as the roast cooks (this kind of regular monitoring is an example of polling).

Interrupt-oriented systems can become overloaded—if interrupts arrive too quickly, the system may not be able to keep up with them. A human air traffic controller, for example, could easily be overwhelmed by a situation in which too many planes converged in a narrow area.

In networked systems, the network interface contains a small amount of memory in which it stores each packet of data that it receives from other computers.



Operating Systems Thinking

Asynchronism vs. Synchronism

When we say events occur asynchronously with the operation of a process, we mean that they happen independently of what is going on in the process. I/O operations can proceed concurrently and asynchronously with an executing process. Once the process initiates an asynchronous I/O operation, the process can continue executing while the I/O

operation proceeds. When the I/O completes, the process is notified. That notification can come at any time. The process can deal with it at that moment or can proceed with other tasks and deal with the I/O-completion interrupt at an appropriate time. So interrupts are often characterized as an asynchronous mechanism. Polling is a synchronous mechanism. The

processor repeatedly tests a device until the I/O is complete. Synchronous mechanisms can spend a lot of time waiting or retesting a device until an event occurs. Asynchronous mechanisms can proceed with other work and waste no time testing for events that have not happened, which generally improves performance.

Each time the network interface receives a packet, it generates an interrupt to inform a processor that data is ready for processing. If a processor cannot process data from the network interface before the interface's memory fills, packets might be lost. Systems typically implement queues to hold interrupts to be processed when a processor becomes available. These queues, of course, consume memory that is limited in size. Under heavy load, the system might not be able to enqueue all arriving interrupts, meaning that some could be lost.

Self Review

1. What does it mean for an interrupt to be synchronous?
2. What is an alternative to interrupts and why is it rarely used?

Ans: 1) A synchronous interrupt occurs due to software execution. 2) A system can perform polling, in which the processor periodically checks the status of devices. This technique is rarely used, because it creates significant overhead when the processor polls devices whose status has not changed. Interrupts eliminate this overhead by notifying a processor only when a device's status changes.

3.4.1 Interrupt Processing

We now consider how computer systems typically process hardware interrupts. (Note that there are other interrupt schemes.)

1. The interrupt line, an electrical connection between the mainboard and a processor, becomes active—devices such as timers, peripheral cards and controllers send signals that activate the interrupt line to inform a processor that an event has occurred (e.g., a period of time has passed or an I/O request has completed). Most processors contain an interrupt controller that orders interrupts according to their priority so that important interrupts are serviced first. Other interrupts are queued until all higher-priority interrupts have been serviced.
2. After the interrupt line becomes active, the processor completes execution of the current instruction, then pauses the execution of the current process. To pause process execution, the processor must save enough information so that the process can be resumed at the correct place and with the correct register information. In early IBM systems, this data was contained in a data structure called the program status word (PSW). In the Intel IA-32 architecture, such process state is referred to as the task state segment (TSS). The TSS is typically stored in a process's PCB.²²
3. The processor then passes control to the appropriate interrupt handler. Each type of interrupt is assigned a unique value that the processor uses as an index into the **interrupt vector**, which is an array of pointers to interrupt handlers. The interrupt vector is located in memory that processes cannot access, so that errant processes cannot modify its contents.
4. The interrupt handler performs appropriate actions based on the type of interrupt.

5. After the interrupt handler completes, the state of the interrupted process (or some other "next process" if the kernel initiates a context switch) is restored.
6. The interrupted process (or some other "next process") executes. It is the responsibility of the operating system to determine whether the interrupted process or some other "next process" executes. This important decision, which can significantly impact the level of service each application receives, is discussed in Chapter 8, Processor Scheduling. For example, if the interrupt signaled an I/O completion event that caused a high-priority process to transition from *blocked* to *ready*, the operating system might preempt the interrupted process and dispatch the high-priority process.

Let us consider how the operating system and hardware interact in response to clock interrupts (Fig. 3.7). At each timer interval, the interrupting clock generates an interrupt that allows the operating system to execute to perform system management operations such as process scheduling. In this case, the processor is executing process P_1 (1) when the clock issues an interrupt (2). Upon receiving the interrupt, the processor accesses the interrupt vector entry that corresponds to the timer interrupt (3).

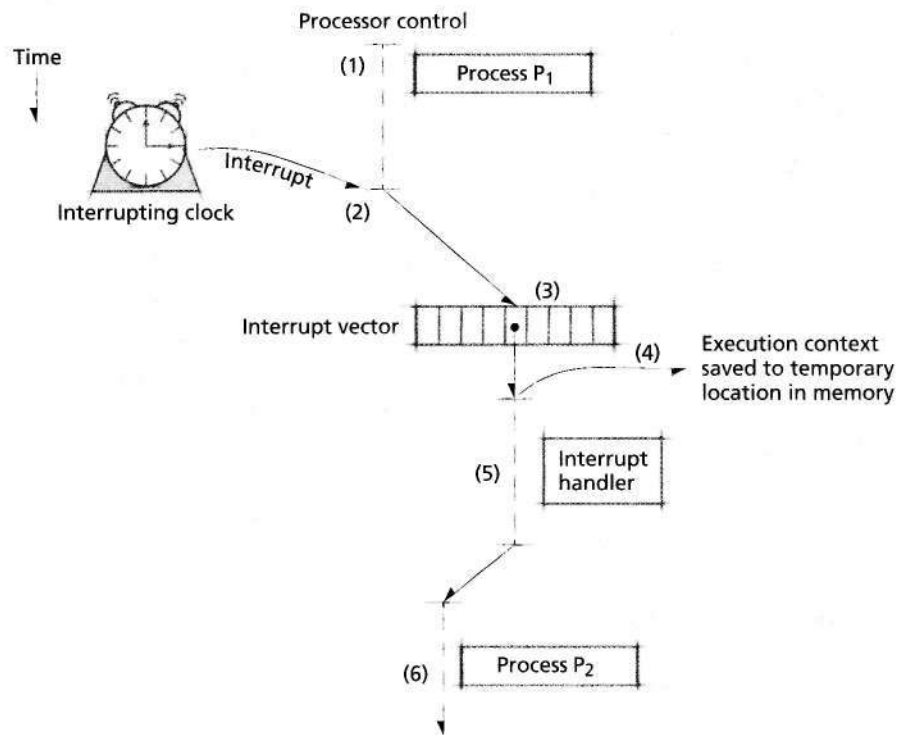


Figure 3.7 | Handling interrupts.

The processor then saves the process's execution context to memory (4) so that the P_1 's execution context is not lost when the interrupt handler executes.²³ The processor then executes the interrupt handler, which determines how to respond to the interrupt (5). The interrupt handler may then restore the state of the previously executing process (P_1) or call the operating system processor scheduler to determine the "next" process to run. In this case, the handler calls the process scheduler, which decides that process P_2 , the highest-priority waiting process, should obtain the processor (6). The context for process P_2 is then loaded from its PCB in main memory, and process P_1 's execution context is saved to its PCB in main memory.

Self Review

1. Why are the locations of interrupt handlers generally not stored in a linked list?
2. Why is the process's execution context saved to memory while the interrupt handler executes?

Ans: **1)** To avoid becoming overwhelmed by interrupts, the system must be able to process each interrupt quickly. Traversing a linked list could significantly increase a system's response time if the number of interrupt types were large. Therefore, most systems use an interrupt vector (i.e., an array) to quickly access the location of an interrupt handler. **2)** If the process's execution context is not saved in memory, the interrupt handler could overwrite the process's registers.

3.4.2 Interrupt Classes

The set of interrupts a computer supports is dependent on the system's architecture. Several types of interrupts are common to many architectures; in this section we discuss the interrupt structure supported by the Intel IA-32 specification,²⁴ which is implemented in Intel® Pentium® processors. (Intel produced over 80 percent of the personal computer processors shipped in 2002.²⁵)

The IA-32 specification distinguishes between two types of signals a processor may receive: interrupts and exceptions. **Interrupts** notify the processor that an event has occurred (e.g., a timer interval has passed) or that an external device's status has changed (e.g., an I/O completion). The IA-32 architecture also provides **software-generated interrupts**—processes can use these to perform system calls. **Exceptions** indicate that an error has occurred, either in hardware or as a result of a software instruction. The IA-32 architecture also uses exceptions to pause a process when it reaches a breakpoint in code.²⁶

Devices that generate interrupts, typically in the form of I/O signals and timer interrupts, are external to a processor. These interrupts are asynchronous with the running process, because they occur independently of instructions being executed by the processor. Software-generated interrupts, such as system calls, are synchronous with the running process, because they are generated in response to an instruction. Figure 3.8 lists several types of interrupts recognized by the IA-32 architecture.

Interrupt Type Description of Interrupts in Each Type

I/O	These are initiated by the input/output hardware. They notify a processor that the status of a channel or device has changed. I/O interrupts are caused when an I/O operation completes, for example.
Timer	A system may contain devices that generate interrupts periodically. These interrupts can be used for tasks such as timekeeping and performance monitoring. Timers also enable the operating system to determine if a process's quantum has expired.
Interprocessor interrupts	These interrupts allow one processor to send a message to another in a multiprocessor system.

Figure 3.8 | Common interrupt types recognized in the Intel IA-32 architectures.

The IA-32 specification classifies exceptions as **faults**, **traps** or **aborts** (Fig. 3.9). Faults and traps are exceptions to which an exception handler can respond to allow processes to continue execution. A fault indicates an error that an exception handler can correct. For example, a page fault occurs when a process attempts to access data that is not in memory (we discuss page faults in Chapter 10, Virtual Memory Organization, and in Chapter 11, Virtual Memory Management). The operating system can correct this error by placing the requested data in main memory. After the problem is corrected, the processor restarts the process that caused the error at the instruction that caused the exception.

Traps do not typically correspond to correctable errors, but rather to conditions such as overflows or breakpoints. For example, as a process instructs a processor to increment the value in an accumulator, the value might exceed the capacity

Exception Class Description of Exceptions in Each Class

Fault	These are caused by a wide range of problems that may occur as a program's machine-language instructions are executed. These problems include division by zero, data (being operated upon) in the wrong format, attempt to execute an invalid operation code, attempt to reference a memory location beyond the limits of real memory, attempt by a user process to execute a privileged instruction and attempt to reference a protected resource.
Trap	These are generated by exceptions such as overflow (when the value stored by a register exceeds the capacity of the register) and when program control reaches a breakpoint in code.
Abort	This occurs when the processor detects an error from which a process cannot recover. For example, when an exception-handling routine itself causes an exception, the processor may not be able to handle both errors sequentially. This is called a double-fault exception, which terminates the process that initiated it.

Figure 3.9 | Intel IA-32 exception classes.

of the accumulator. In this case, the operating system can simply notify the process that an overflow occurred. After executing the trap's exception handler, the processor restarts the process at the next instruction following the one that caused the exception.

Aborts indicate errors from which the process (or perhaps even the system) cannot recover, such as hardware failure. In this case, the processor cannot reliably save the process's execution context. Typically, as a result, the operating system terminates prematurely the process that caused the abort.

Most architectures and operating systems prioritize interrupts, because some require more immediate action than others. For example, responding to a hardware failure is more important than responding to an I/O-completion event. Interrupt priorities can be implemented in both hardware and software simultaneously. For example, a processor might block or queue interrupts of a lower priority than that of the interrupt the processor is currently handling. At times, the kernel can become so overloaded with interrupts that it can no longer respond to them. Rapid response to interrupts and quick return of control to interrupted processes is essential to maximizing resource utilization and achieving a high degree of interactivity. Most processors therefore allow the kernel to **disable** (or **mask**) an interrupt type. The processor may then ignore interrupts of that type or store them in a queue of pending interrupts that are delivered when that type of interrupt is reenabled. In the IA-32 architecture, the processor provides a register that indicates whether interrupts are disabled.²⁷

Self Review

1. In the IA-32 architecture, what two types of signals can a processor receive?
2. In the IA-32 architecture, what is the difference between a fault and a trap?

Ans: 1) A processor can receive interrupts or exceptions. Interrupts indicate that an event has occurred; exceptions indicate that an error has occurred. 2) A fault restarts a process from the instruction that caused the exception. Faults are generally errors that can be corrected. A trap restarts a process at the next instruction following the one that caused the exception. Traps are usually generated by system calls and by the arrival of program control at breakpoints.

3.5 Interprocess Communication

In multiprogrammed and networked environments, it is common for processes to communicate with one another. Many operating systems provide mechanisms for interprocess communication (IPC) that, for example, enable a text editor to send a document to a print spooler or a Web browser to retrieve data from a distant server. Interprocess communication is also essential for processes that must coordinate (i.e., synchronize) activities to achieve a common goal. The case studies on Linux (see Section 20.10, Interprocess Communication) and Windows XP (see Section 21.10, Interprocess Communication) discuss how IPC is implemented in popular operating systems.

3.5.1 Signals

Signals are software interrupts that notify a process that an event has occurred. Unlike other IPC mechanisms we discuss, signals do not allow processes to specify data to exchange with other processes.²⁸ A system's signals depend on the operating system and the software-generated interrupts supported by a particular processor. When a signal occurs, the operating system first determines which process should receive the signal and how that process will respond to the signal.

Processes may catch, ignore or mask a signal. A process **catches** a signal by specifying a routine that the operating system calls when it delivers the signal.²⁹ A process may also **ignore** the signal. In this case, the process relies on the operating system's **default action** to handle the signal. A common default action is to **abort**, which causes the process to exit immediately. Another common default action is called a **memory dump**, which is similar to aborting. A memory dump causes a process to exit, but before doing so, the process generates a **core file** that contains the process's execution context and data from its address space, which is useful for debugging. A third default action is to simply ignore the signal. Two other default actions are to suspend and, subsequently, resume a process.³⁰

A process can also block a signal by **masking** it. When a process masks a signal of a specific type (e.g., the suspend signal), the operating system does not deliver signals of that type until the process clears the signal mask. Processes typically block a signal type while handling another signal of the same type. Similar to masked interrupts, masked signals may be lost, depending on the operating system implementation.

Self Review

1. What is the major drawback of using signals for IPC?
2. What are the three ways in which a process can respond to a signal?

Ans: 1) Signals do not support data exchange between processes. 2) A process can catch, ignore or mask a signal.

3.5.2 Message Passing

With the increasing prominence of distributed systems, there has been a surge of interest in message-based interprocess communication.^{31, 32, 33, 34, 35, 36} We discuss message-based communication in this section; particular implementations are discussed in the Linux and Windows XP case studies.^{37, 38}

Messages can be passed in one direction at a time—for any given message, one process is the sender and the other is the receiver. Message passing may be bidirectional, meaning that each process can act as either a sender or a receiver while participating in interprocess communication. One model of message passing specifies that processes send and receive messages by making calls such as

```
send( receiverProcess, message );
receive( senderProcess, message );
```

The send and receive calls are normally implemented as system calls accessible from many programming language environments. A **blocking send** must wait for the receiver to receive the message, requiring that the receiver notify the sender when the message is received (this notification is called an acknowledgment). A **nonblocking send** enables the sender to continue with other processing even if the receiver has not yet received (and acknowledged) the message; this requires a message buffering mechanism to hold the message until the receiver receives it. A blocking send is an example of **synchronous communication**; a nonblocking send is an example of **asynchronous communication**. The send call may explicitly name a receiving process, or it may omit the name, indicating that the message is to be **broadcast** to all processes (or to some "working group" with which the sender generally communicates).

Asynchronous communication with nonblocking sends increases throughput by reducing the time that processes spend waiting. For example, a sender may send information to a busy print server; the system will buffer this information until the print server is ready to receive it, and the sender will continue execution without having to wait on the print server.

If no message has been sent, then a blocking receive call forces the receiver to wait; a nonblocking receive call enables the receiver to continue with other processing before it next attempts a receive. A receive call may specify that a message is to be received from a particular sender, or the receive may receive a message from any sender (or from any member of a group of senders).

A popular implementation of message passing is a **pipe**—a region of memory protected by the operating system that serves as a buffer, allowing two or more processes to exchange data. The operating system synchronizes access to the buffer—after a writer completes writing to the buffer (possibly filling it), the operating system pauses the writer's execution and allows a reader to read data from the buffer. As a process reads data, that data is removed from the pipe. When the reader completes reading data from the buffer (possibly emptying it), the operating system pauses the reader's execution and allows the writer to write data to the buffer.³⁹ Detailed treatments of pipes are provided in the Linux and Windows XP case studies at the end of the book. See Section 20.10.2, Pipes, and Section 21.10.1, Pipes, respectively.

In our discussions of interprocess communication between processes on the same computer, we always assumed flawless transmission. In distributed systems, on the other hand, transmissions can be flawed and even lost. So senders and receivers often cooperate using an **acknowledgment protocol** for confirming that each transmission has been properly received. A timeout mechanism can be used by the sender waiting for an acknowledgment message from the receiver; on timeout, if the acknowledgment has not been received, the sender can retransmit the message. Message passing systems with retransmission capabilities can identify each new message with a sequence number. The receiver can examine these numbers to be sure that it has received every message and to resequence out-of-sequence messages. If an acknowledgment message is lost and the sender decides to retransmit, it assigns the same sequence number to the retransmitted message as to

the originally transmitted one. The receiver detecting several messages with the same sequence number knows to keep only one of them.

One complication in distributed systems with send/receive message passing is in naming processes unambiguously so that explicit send and receive calls reference the proper processes. Process creation and destruction can be coordinated through some centralized naming mechanism, but this can introduce considerable transmission overhead as individual machines request permission to use new names. An alternate approach is to have each computer ensure unique process names for its own processes; then processes may be addressed by combining the computer name with the process name. This, of course, requires centralized control in determining a unique name for each computer in a distributed system, which could potentially incur significant overhead if computers are frequently added and removed from the network. In practice, distributed systems pass messages between computers using numbered ports on which processes listen, avoiding the naming problem (see Chapter 16, Introduction to Networking).

As we will see in Chapter 17, Introduction to Distributed Systems, message-based communication in distributed systems presents serious security problems. One of these is the **authentication problem**: How do the senders and receivers know that they are not communicating with imposters who may be trying to steal or corrupt their data? Chapter 19, Security, discusses several authentication approaches.

There are several IPC techniques that we discuss later in the book. In addition to signals and pipes, processes may communicate via shared memory (discussed in Chapter 10, Virtual Memory Organization), sockets (discussed in Chapter 16, Introduction to Networking) and remote procedure calls (discussed in Chapter 17). They also may communicate to synchronize activities using semaphores and monitors, which are discussed in Chapter 5, Asynchronous Concurrent Execution, and Chapter 6, Concurrent Programming, respectively.

Self Review

1. Why do distributed systems rely on message passing instead of signals?
2. When a process performs a blocking send, it must receive an acknowledgment message to unblock. What problem might result from this scheme, and how can it be avoided?

Ans 1) Signals are typically architecture specific, meaning that signals supported by one computer may not be compatible with signals supported by another. Also, signals do not allow processes to transmit data, a capability required by most distributed systems. 2) The sender may never receive an acknowledgment message, meaning that the process could be blocked indefinitely. This can be remedied by a timeout mechanism—if the sender does not receive an acknowledgment after a period of time, the send operation is assumed to have failed and it can be retried.

3.6 Vase Study: UNIX Processes

UNIX and UNIX-based operating systems provide an implementation of processes that has been borrowed by many other operating systems (see the Mini Case Study,

UNIX Systems). In this section, we describe the structure of UNIX processes, discuss several UNIX features that motivate the discussion in the following chapters and introduce how UNIX allows users to perform process management operations.

Each process must store its code, data and stack in memory during execution. In a real memory system, processes would locate such information by referencing a range of physical addresses. The range of valid main memory addresses for each process is determined by the size of main memory and the memory consumed by other processes. Because UNIX implements virtual memory, all UNIX processes are provided with a set of memory addresses, called a **virtual address space**, in which the process may store information. The virtual address space contains a text region, data region and stack region.⁴⁰

The kernel maintains a process's PCB in a protected region of memory that user processes cannot access. In UNIX systems, a PCB stores information including the contents of processor registers, the process identifier (PID), the program counter and the system stack.⁴¹⁻⁴² The PCBs for all processes are listed in the process table, which allows the operating system to access information (e.g., priority) regarding every process.⁴³

UNIX processes interact with the operating system via system calls. Figure 3.10 lists several of these. A process can spawn a child process by using the **fork** system call, which creates a copy of the parent process.⁴⁴⁻⁴⁵ The child process receives a copy of the parent process's data and stack segments and any other resources.⁴⁶⁻⁴⁷ The text segment, which contains the parent's read-only instructions, is shared with its child. Immediately following the fork, the parent and child process contain identical data and instructions. This means that the two processes must perform exactly the same actions unless either the parent or child can determine its identity. The **fork** system call therefore returns different values; the parent process



Mini Case Study

In the days before Windows, Macintosh, Linux or even DOS, operating systems typically worked on only one model of computer, managing system resources, running batch streams and little more.⁴⁸ From 1965 to 1969, a

group of research teams from Bell Laboratories, General Electric and Project MAC at MIT developed the **Multics** operating system—a general-purpose computer utility, designed to be “all things to all people.”⁴⁹ It was large, expensive

and complex. In 1969, Bell Labs withdrew from the project and their own small team, led by Ken Thompson, began designing a more practical operating system to run machines at Bell Labs. *(Continued on the next page.)*



Mini Case Study

UNIX Systems (Cont.)

Thompson implemented the basic components of the operating system, which Brian Kernighan named UNICS, a joke on the "multi" aspect of Multics; the spelling eventually changed to UNIX. Over the next few years, UNIX was rewritten in an interpreted implementation of Thompson's language B (based on Martin Richard's BCPL programming language), and soon after in Dennis Ritchie's faster, compiled C language.⁵⁰

Due to a federal anti-trust lawsuit, AT&T (which owned Bell Labs) was not allowed to sell computer products, so they distributed UNIX source code to universities for a small fee to cover just the expense of producing the magnetic tapes. A group of students at the University of California at Berkeley, led by Bill Joy (later a cofounder of Sun Microsystems), modified the UNIX source code, evolving the operating system into what became known as **Berkeley Software Distribution UNIX (BSD UNIX)**.⁵¹

Industry software developers were drawn to UNIX because it was free, small and customize-

able. To work with UNIX, developers had to learn C, and they liked it. Many of these developers also taught in colleges, and C gradually replaced Pascal as the preferred teaching language in college programming courses. Sun Microsystems based its SunOS on BSD UNIX, then later teamed up with AT&T to design the **Solaris** operating system based on AT&T's System V Release 4 UNIX.⁵² A group of other UNIX developers, concerned that Sun's association with AT&T would give Sun an unfair business lead over other UNIX developers, formed the **Open Software Foundation (OSF)** to produce their own non-proprietary version of UNIX called **OSF/1**; the fierce competition between OSF and AT&T-backed Sun was dubbed the UNIX Wars.⁵³

Several important operating systems are based on UNIX technology. Professor Andrew Tanenbaum of the Vrije Universiteit in Amsterdam built Minix in 1987, a stripped-down version of UNIX that was designed for teaching OS basics and is still used for this purpose in some college courses. Linus Torvalds, a Finnish graduate student, used Minix to

begin writing the well-known open-source Linux operating system—now a whole family of systems in its own right (see Chapter 20, Case Study: Linux).⁵⁴ Linux is the most popular open-source operating system, and companies including IBM, Hewlett-Packard, Sun Microsystems and Intel all offer Linux versions as an operating system option for their servers. OpenBSD is another open-source project, led by Theo de Raadt, and is recognized as the most secure operating system available (see Chapter 19, Security).^{55, 56, 57, 58} FreeBSD is also open-source and is known for its ease of use.⁵⁹ Yet another BSD descendant, NetBSD, has focused on portability to a variety of systems.^{60, 61} IBM's AIX, based on both System V and BSD,⁶² runs on some of IBM's servers. IBM claims AIX has a high degree of source-code compatibility with Linux.⁶³ Hewlett-Packard's HP-UX is becoming a strong competitor to AIX and Solaris, achieving the highest ratings in all the categories in a 2002 D.H. Brown Associates report, placing ahead of both Solaris and **AIX**.^{64, 65, 66}

receives the PID of the child and the child process receives a value of zero. This convention allows the child process to recognize that it is newly created. Application programmers can use this convention to specify new instructions for the child process to execute.

A process can call `exec` to load a new program from a file; `exec` is often performed by a child process immediately after it is spawned.⁶⁷ When the parent creates a child process, the parent can issue a `wait` system call, which blocks the parent until the specified child process terminates.⁶⁸ After a process has completed its work, it issues the `exit` system call. This tells the kernel that the process has finished; the kernel responds by freeing all of the process's memory and other resources, such as open files. When a parent process exits, its child processes are typically relocated in the process hierarchy to be children of the *init* process.^{69,70} If a parent process is terminated by a `kill` signal from another process, that signal is also sent to its child processes.

UNIX process priorities are integers between -20 and 19 (inclusive) that the system uses to determine which process will run next. A lower numerical priority value indicates a higher scheduling priority.⁷¹ Processes that belong to the operating system, called kernel processes, often have negative integer values and typically have higher scheduling priority than user processes.⁷² Operating system processes that perform maintenance operations periodically, called daemons, typically execute with the lowest possible priority.

Many applications require several independent components to communicate during execution, requiring interprocess communication (IPC). UNIX provides several mechanisms to enable processes to exchange data, such as signals and pipes (see Section 3.5, Interprocess Communication, and Section 20.10.2, Pipes).⁷³

<i>System Call</i>	<i>Description</i>
<code>fork</code>	Spawns a child process and allocates to that process a copy of its parent's resources.
<code>exec</code>	Loads a process's instructions and data into its address space from a file.
<code>wait</code>	Causes the calling process to block until its child process has terminated.
<code>signal</code>	Allows a process to specify a signal handler for a particular signal type.
<code>exit</code>	Terminates the calling process.
<code>nice</code>	Modifies a process's scheduling priority.

Figure 3.10 | UNIX system calls.

136 Process Concepts

Self Review

1. Why can a parent and its child share the parent's text segment after a fork system call?
2. Why must a process use IPC to share data with other processes?

Ans: 1) The text segment contains instructions that cannot be modified by either process, meaning that both the parent and child process will execute the same instructions regardless of whether the operating system maintains one, or multiple, copies of the segment in memory. Therefore, the operating system can reduce memory consumption by sharing access to the text region between a parent and its child. **2)** The operating system does not allow unrelated processes to share the data segment of their address spaces, meaning that data stored by one process is inaccessible to an unrelated process. Therefore, the operating system must provide some mechanism to make data from one process available to another.

Web Resources

msdn.microsoft.com/library/en-us/dllproc/base/about_processes_and_threads.asp

Provides a description of processes in Windows XP.

www.freebsd.org/handbook/basics-processes.html

Includes a description of how the FreeBSD operating system handles processes. Mac OS X is based (in part) on FreeBSD.

www.linux-tutorial.info/cgi-bin/display.pl?83&99980&0&3

Discusses how Linux handles processes.

docs.sun.com/db/doc/806-4125/6jd7pe6bg?a=view

Provides a process state-transition diagram for Sun Microsystems's Solaris operating system.

www.beyondlogic.org/interrupts/interrupt.htm

Overviews interrupts and provides a detailed description of interrupt handling in the Intel architecture.

developer.apple.com/documentation/Hardware/Device-Managers/pci_srvcs/pci_cards_drivers/PCI_B00K.16d.html

Documents the interrupt model in Apple Macintosh computers.

Summary

A process, which is a program in execution, is central to understanding how today's computer systems perform and keep track of many simultaneous activities. Each process has its own address space, which may consist of a text region, data region and stack region. A process moves through a series of discrete process states. For example, a process can be in the *running* state, *ready* state or *blocked* state. The ready list and blocked list store references to processes that are not *running*.

When a process reaches the head of the ready list, and when a processor becomes available, that process is given the processor and is said to make a transition from the *ready* state to the *running* state. The act of assigning a processor to the first process on the ready list is called dispatching. To prevent any one process from monopolizing the system, either accidentally or maliciously, the operating system sets a hardware interrupting clock (or interval timer) to allow a process to run for a specific time interval

or quantum. If a *running* process initiates an input/output operation before its quantum expires, the process is said to block itself pending the completion of the I/O operation. Alternatively, the operating system can employ cooperative multitasking in which each process runs until completion or until it voluntarily relinquishes its processor. This can be dangerous, because cooperative multitasking does not prevent processes from monopolizing a processor.

The operating system typically performs several operations when it creates a process, including assigning a process identification number (PID) to the process and creating a process control block (PCB), or process descriptor, which stores the program counter (i.e., the pointer to the next instruction the process will execute), PID, scheduling priority and the process's execution context. The operating system maintains pointers to each process's PCB in the process table so that it can access PCBs quickly. When a process terminates (or is terminated by the operating sys-

tern), the operating system removes the process from the process table and frees all of the process's resources, including its memory.

A process may spawn a new process—the creating process is called the parent process and the created process is called the child process. Exactly one parent process creates a child. Such creation yields a hierarchical process structure. In some systems, a spawned process is destroyed automatically when its parent is destroyed; in other systems, spawned processes proceed independently of their parents, and the destruction of a parent has no effect on the destroyed parent's children.

A suspended process is indefinitely removed from contention for time on the processor without being destroyed. The suspended states are *suspendedready* and *suspendedblocked*. A suspension may be initiated by the process being suspended or by another process; a suspended process must be resumed by another process.

When the operating system dispatches a ready process to a processor, it initiates a context switch. Context switches must be transparent to processes. During a context switch a processor cannot perform any "useful" computation, so operating systems must minimize context-switching time. Some architectures reduce overhead by performing context-switching operations in hardware.

Interrupts enable software to respond to signals from hardware. An interrupt may be specifically initiated by a running process (in which case it is often called a trap and said to be synchronous with the operation of the process), or it may be caused by some event that may or may not be related to the running process (in which case it is said to be asynchronous with the operation of the process). An alternative to interrupts is for the processor to repeatedly request the status of each device, an approach called polling.

Interrupts are essential to maintaining a productive and protected computing environment. When an interrupt occurs, the processor will execute one of the kernel's interrupt-handling functions. The interrupt handler determines how the system should respond to interrupts. The locations

of the interrupt handlers are stored in an array of pointers called the interrupt vector. The set of interrupts a computer supports depends on the system's architecture. The IA-32 specification distinguishes between two types of signals a processor may receive: interrupts and exceptions.

Many operating systems provide mechanisms for interprocess communication (IPC) that, for example, enable a Web browser to retrieve data from a distant server. Signals are software interrupts that notify a process that an event has occurred. Signals do not allow processes to specify data to exchange with other processes. Processes may catch, ignore or mask a signal.

Message-based interprocess communication can occur in one direction at a time or it may be bidirectional. One model of message passing specifies that processes send and receive messages by making calls. A popular implementation of message passing is a pipe—a region of memory protected by the operating system that allows two or more processes to exchange data. One complication in distributed systems with send/receive message passing is in naming processes unambiguously so that explicit send and receive calls reference the proper processes.

UNIX processes are provided with a set of memory addresses, called a virtual address space, which contains a text region, data region and stack region. In UNIX systems, a PCB stores information including the contents of processor registers, the process identifier (PID), the program counter and the system stack. All processes are listed in the process table, which allows the operating system to access information regarding every process. UNIX processes interact with the operating system via system calls. A process can spawn a child process by using the fork system call, which creates a copy of the parent process. UNIX process priorities are integers between -20 and 19 (inclusive) that the system uses to determine which process will run next; a lower numerical priority value indicates a higher scheduling priority. The kernel also provides IPC mechanisms, such as pipes, to allow unrelated processes to transfer data.

Key Terms

abort—Action that terminates a process prematurely. Also, in the IA-32 specification, an error from which a process cannot recover.

Access Control List (ACL) (Multics)—Multics' discretionary access control implementation.

Access Isolation Mechanism (AIM) (Multics)—Multics' mandatory access control implementation.

address space—Set of memory locations a process can reference.

138 Process Concepts

Berkeley Software Distribution (BSD) UNIX—UNIX version modified and released by a team led by Bill Joy at the University of California at Berkeley. BSD UNIX is the parent of several UNIX variations.

blocked state—Process state in which the process is waiting for the completion of some event, such as an I/O completion, and cannot use a processor even if one is available.

blocked list—Kernel data structure that contains pointers to all *blocked* processes. This list is not maintained in any particular priority order.

child process—Process that has been spawned from a parent process. A child process is one level lower in the process hierarchy than its parent process. In UNIX systems, child processes are created using the fork system call.

concurrent program execution—Technique whereby processor time is shared among multiple active processes. On a uni-processor system, concurrent processes cannot execute simultaneously; on a multiprocessor system, they can.

context switching—Action performed by the operating system to remove a process from a processor and replace it with another. The operating system must save the state of the process that it replaces. Similarly, it must restore the state of the process being dispatched to the processor.

cooperative multitasking—Process scheduling technique in which processes execute on a processor until they voluntarily relinquish control of it.

data region—Section of a process's address space that contains data (as opposed to instructions). This region is modifiable.

disable (mask) interrupts—When a type of interrupt is disabled (masked), interrupts of that type are not delivered to the process that has disabled (masked) the interrupts. The interrupts are either queued to be delivered later or dropped by the processor.

dispatcher—Operating system component that assigns the first process on the ready list to a processor.

exception—Hardware signal generated by an error. In the Intel IA-32 specification, exceptions are classified as traps, faults and aborts.

fault—In the Intel IA-32 specification, an exception as the result of an error such as division by zero or illegal access to memory. Some faults can be corrected by appropriate operating system exception handlers.

I/O completion interrupt—Message issued by a device when it finishes servicing an I/O request.

hierarchical process structure—Organization of processes when parent processes spawn child processes and, in particular, only one parent creates a child.

interrupt—Hardware signal indicating that an event has occurred. Interrupts cause the processor to invoke a set of software instructions called an interrupt handler.

interrupt handler—Kernel code that is executed in response to an interrupt.

interrupt vector—Array in protected memory containing pointers to the locations of interrupt handlers.

interrupting clock (interval timer)—Hardware device that issues an interrupt after a certain amount of time (called a quantum), e.g., to prevent a process from monopolizing a processor.

message passing—Mechanism to allow unrelated processes to communicate by exchanging data.

Multics Relational Data Store (MRDS)—First commercial relational database system, included in Multics.

Open Software Foundation (OSF)—Coalition of UNIX developers that built the OSF/1 UNIX clone to compete with AT&T's and Sun's Solaris. The OSF and the AT&T/Sun partnership were the participants in the UNIX Wars.

OSF/1—UNIX clone built by the Open Software Foundation to compete with Solaris.

parent process—Process that has spawned one or more child processes. In UNIX, this is accomplished by issuing a fork system call.

pipe—Message passing mechanism that creates a direct data stream between two processes.

polling—Technique to discover hardware status by repeatedly testing each device. Polling can be implemented in lieu of interrupts but typically reduces performance due to increased overhead.

process—Entity that represents a program in execution.

process control block (PCB)—Data structure containing information that characterizes a process (e.g., PID, address space and state); also called a process descriptor.

process descriptor—See process control block (PCB).

process identification number (PID)—Value that uniquely identifies a process.

process priority—Value that determines the importance of a process relative to other processes. It is often used to determine how a process should be scheduled for execution on a processor relative to other processes.

process state—Status of a process (e.g., *running*, *ready*, *blocked*, etc.).

process table—Data structure that contains pointers to all processes in the system.

program counter—Pointer to the instruction a processor is executing for a running process. After the processor completes the instruction, the program counter is adjusted to point to the next instruction the processor should execute.

quantum—Unit of time during which a process can execute before it is removed from the processor. Helps prevent processes from monopolizing processors.

ready state—Process state from which a process may be dispatched to the processor.

ready list—Kernel data structure that organizes all *ready* processes in the system. The ready list is typically ordered by process scheduling priority.

resume—Remove a process from a suspended state.

running state—Process state in which a process is executing on a processor.

signal—Message sent by software to indicate that an event or error has occurred. Signals cannot pass data to their recipients.

Solaris—UNIX version based on both System V Release 4 and SunOS, designed by AT&T and Sun collaboratively.

spawning a process—A parent process creating a child process.

stack region—Section of process's address space that contains instructions and values for open procedure calls. The contents of the stack grow as a process issues nested procedure calls and shrink as called procedures return.

state transition—Change of a process from one state to another.

suspended state—Process state (either *suspendedblocked* or *suspendedready*) in which a process is indefinitely removed from contention for time on a processor without being destroyed. Historically, this operation allowed a system operator to manually adjust the system load and/or respond to threats of system failure.

suspendedblocked state—Process state resulting from the process being suspended while in the *blocked* state. Resuming such a process places it into the *blocked* state.

suspendedready state—Process state resulting from the process being suspended while in the *ready* state. Resuming such a process places it into the *ready* state.

text region—Section of a process's address space that contains instructions that are executed by a processor.

trap—In the IA-32 specification, an exception generated by an error such as overflow (when the value stored by a register exceeds the capacity of the register). Also generated when program control reaches a breakpoint in code.

unblock—Remove a process from the *blocked* state after the event on which it was waiting has completed.

virtual address space—Set of memory addresses that a process can reference. A virtual address space may allow a process to reference more memory than is physically available in the system.

Exercises

3.1 Give several definitions of process. Why, do you suppose, is there no universally accepted definition?

3.2 Sometimes the terms user and process are used interchangeably. Define each of these terms. In what circumstances do they have similar meanings?

```

1  int  main() {
2
3      while( true ) {
4          fork();
5      }
6
7  }
```

Figure 3.1 | Code for Exercise 3.4.

3.3 Why does it not make sense to maintain the blocked list in priority order?

3.4 The ability of one process to spawn a new process is an important capability, but it is not without its dangers. Consider the consequences of allowing a user to run the process in Fig.3.11. Assume that `fork()` is a system call that spawns a child process.

140 Process Concepts

- a. Assuming that a system allowed such a process to run, what would the consequences be?
- b. Suppose that you as an operating systems designer have been asked to build in safeguards against such processes. We know (from the "Halting Problem" of computability theory) that it is impossible, in the general case, to predict the path of execution a program will take. What are the consequences of this fundamental result from computer science on your ability to prevent processes like the above from running?
- c. Suppose you decide that it is inappropriate to reject certain processes, and that the best approach is to place certain runtime controls on them. What controls might the operating system use to detect processes like the above at runtime?
- d. Would the controls you propose hinder a process's ability to spawn new processes?
- e. How would the implementation of the controls you propose affect the design of the system's process handling mechanisms?

3.5 In single-user dedicated systems, it is generally obvious when a program goes into an infinite loop. But in multiuser systems running large numbers of processes, it cannot easily be determined that an individual process is not progressing.

- a. Can the operating system determine that a process is in an infinite loop?
- b. What reasonable safeguards might be built into an operating system to prevent processes in infinite loops from running indefinitely?

3.6 Choosing the correct quantum size is important to the effective operation of an operating system. Later in the text we will consider the issue of quantum determination in depth. For now, let us anticipate some of the problems.

Consider a single-processor timesharing system that supports a large number of interactive users. Each time a process gets the processor, the interrupting clock is set to interrupt after the quantum expires. This allows the operating system to prevent any single process from monopolizing the processor and to provide rapid responses to interactive processes. Assume a single quantum for all processes on the system.

- a. What would be the effect of setting the quantum to an extremely large value, say 10 minutes?
- b. What if the quantum were set to an extremely small value, say a few processor cycles?
- c. Obviously, an appropriate quantum must be between the values in (a) and (b). Suppose you could turn a dial and vary the quantum, starting with a small value

and gradually increasing. How would you know when you had chosen the "right" value?

- d. What factors make this value right from the user's standpoint?
- e. What factors make it right from the system's standpoint?

3.7 In a block/wakeup mechanism, a process blocks itself to wait for an event to occur. Another process must detect that the event has occurred, and wake up the blocked process. It is possible for a process to block itself to wait for an event that will never occur.

- a. Can the operating system detect that a blocked process is waiting for an event that will never occur?
- b. What reasonable safeguards might be built into an operating system to prevent processes from waiting indefinitely for an event?

3.8 One reason for using a quantum to interrupt a running process after a "reasonable" period is to allow the operating system to regain the processor and dispatch the next process. Suppose that a system does not have an interrupting clock, and the only way a process can lose the processor is to relinquish it voluntarily. Suppose also that no dispatching mechanism is provided in the operating system.

- a. Describe how a group of user processes could cooperate among themselves to effect a user-controlled dispatching mechanism.
- b. What potential dangers are inherent in this scheme?
- c. What are the advantages to the users over a system-controlled dispatching mechanism?

3.9 In some systems, a spawned process is destroyed automatically when its parent is destroyed; in other systems, spawned processes proceed independently of their parents, and the destruction of a parent has no effect on its children.

- a. Discuss the advantages and disadvantages of each approach.
- b. Give an example of a situation in which destroying a parent should specifically not result in the destruction of its children.

3.10 When interrupts are disabled on most devices, they remain pending until they can be processed when interrupts are again enabled. No further interrupts are allowed. The functioning of the devices themselves is temporarily halted. But in real-time systems, the environment that generates the interrupts is often disassociated from the computer system. When interrupts are disabled on the computer system, the environment keeps on generating interrupts anyway. These interrupts can be lost.

- a. Discuss the consequences of lost interrupts.
- b. In a real-time system, is it better to lose occasional interrupts or to halt the system temporarily until interrupts are again enabled?

3.11 As we will see repeatedly throughout this text, management of waiting is an essential part of every operating system. In this chapter we have seen several waiting states, namely *ready*, *blocked*, *suspendedready* and *suspendedblocked*. For each of these states discuss how a process might get into the state, what the process is waiting for, and the likelihood that the process could get "lost" waiting in the state indefinitely.

What features should operating systems incorporate to deal with the possibility that processes could start to wait for an event that might never happen?

3.12 Waiting processes do consume various system resources. Could someone sabotage a system by repeatedly creating pro-

cesses and making them wait for events that will never happen? What safeguards could be imposed?

3.13 Can a single-processor system have no processes ready and no process running? Is this a "dead" system? Explain your answer.

3.14 Why might it be useful to add a *dead* state to the state-transition diagram?

3.15 System A runs exactly one process per user. System B can support many processes per user. Discuss the organizational differences between operating systems A and B with regard to support of processes.

3.16 Compare and contrast IPC using signals and message passing.

3.17 As discussed in Section 3.6, Case Study: UNIX Processes, UNIX processes may change their priority using the *nice* system call. What restrictions might UNIX impose on using this system call, and why?

Suggested Projects

3.18 Compare and contrast what information is stored in PCBs in Linux, Microsoft's Windows XP, and Apple's OS X. What process states are defined by each of these operating systems?

3.19 Research the improvements made to context switching over the years. How has the amount of time processors spend on context switches improved? How has hardware helped to make context switching faster?

3.20 The Intel Itanium line of processors, which are designed for high-performance computing, are implemented according to the IA-64 (64-bit) specification. Compare and contrast the IA-32 architecture's method of interrupt processing discussed in this chapter with that of the IA-64 architecture (see developer.intel.com/design/itanium/manuals/245318.pdf).

3.21 Discuss an interrupt scheme other than that described in this chapter. Compare the two schemes.

Recommended Reading

Randell and Horning describe basic process concepts as well as management of multiple processes.⁷⁴ Lampson considers some basic process concepts, including context switching.⁷⁵ Quarterman, Silberschatz and Peterson discuss how BSD UNIX 4.3 manages processes.⁷⁶ The case studies in Part 8 describe how Linux and Windows XP implement and manage processes. Information on how processors handle interrupts is available in computer architecture books such as *Computer Organization and Design* by Patterson and Hennessey.⁷⁷

Interprocess communication is a focal point of microkernel (see Section 1.13.3, Microkernel Architecture) and exokernel operating system architectures. Because the performance of such systems relies heavily on the efficient operation of IPC mechanisms, considerable research has been devoted to the topic of improving IPC performance. Early message-based

architectures⁷⁸ such as Chorus were distributed operating systems, but as early as 1986, microkernels such as Mach⁷⁹ were developed. IPC performance improvement to support effective microkernels is reflected in the literature.⁸⁰⁻⁸¹

IPC mechanisms differ from one operating system to another. Most UNIX systems share the mechanisms presented in this chapter. Descriptions of such implementations can be found in the Linux manual pages, but the reader is encouraged to read Chapter 20, Case Study: Linux, first. *Understanding the Linux Kernel*⁸² discusses IPC in Linux and *Inside Windows 2000*⁸³ discusses IPC in Windows 2000 (almost all of which applies to Windows XP as well). The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Daley, Robert C. and Jack B. Dennis, "Virtual Memory, Processes, and Sharing in Multics," *Proceedings of the ACM Symposium on Operating System Principles*, January 1967.
2. Corbato, R; M. Merwin-Daggett, and R.C. Daley, "An Experimental Time-Sharing System," *Proceedings of the Spring Joint Computer Conference (AFIPS)*, Vol. 21, 1962, pp. 335-344.
3. Van Vleck, X, "The IBM 7094 and CTSS," March 3, 2003, <www.multicians.org/thvv/7094.html>.
4. Van Vleck, T, "Multics General Information and FAQ," September 14, 2003, <www.multicians.org/general.html>.
5. Van Vleck, T, "Multics General Information and FAQ," September 14, 2003, <www.multicians.org/general.html>.
6. Green, P., "Multics Virtual Memory: Tutorial and Reflections," 1993, <[ftp://ftp.stratus.com/pub/vos/multics/pg/mvm.html](http://ftp.stratus.com/pub/vos/multics/pg/mvm.html)>.
7. Van Vleck, T, "Multics General Information and FAQ," September 14, 2003, <www.multicians.org/general.html>.
8. Green, P., "Multics Virtual Memory: Tutorial and Reflections," 1993, <[ftp://ftp.stratus.com/pub/vos/multics/pg/mvm.html](http://ftp.stratus.com/pub/vos/multics/pg/mvm.html)>.
9. Van Vleck, T, "Multics General Information and FAQ," September 14, 2003, <www.multicians.org/general.html>.
10. Van Vleck, T, "Multics Glossary—A," <www.multicians.org/mga.html>.
11. Van Vleck, T, "Multics Glossary—B," <www.multicians.org/mgb.html>.
12. McJones, P., "Multics Relational Data Store (MRDS)," <www.mcjones.org/System_R/mrds.html>.
13. Peterson, J. L.; J. S. Quarterman; and A. Silbershatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, p. 388.
14. MIT Laboratory for Computer Science, "Prof. F. J. Corbato," April 7, 2003, <www.lcs.mit.edu/people/bioprint.php3?PeopleID=86>.
15. F. Corbato, M. Merwin-Daggett, and R.C. Daley, "An Experimental Time-Sharing System," *Proc. Spring Joint Computer Conference (AFIPS)*, 335-344.
16. MIT Laboratory for Computer Science, "Prof. F. J. Corbato," April 7, 2003, <www.lcs.mit.edu/people/bioprint.php3?PeopleID=86>.
17. MIT Laboratory for Computer Science, "Prof. F. J. Corbato," April 7, 2003, <www.lcs.mit.edu/people/bioprint.php3?PeopleID=86>.
18. F. Corbato, "On Building Systems That Will Fail," *Communications of the ACM*, Vol. 34, No. 9, September 1991, pp. 72-81.
19. "UNIX System Calls Links," <www.softpanorama.org/Internals/unix_system_calls_links.shtml>.
20. Lampson, B. W., "A Scheduling Philosophy for Multiprocessing System," *Communications of the ACM*, Vol. 11, No. 5, 1968, pp. 347-360.
21. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002, pp. 6-1-6-15.
22. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002.
23. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002, pp. 5-16.
24. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002.
25. Krazit, T, "Study: Intel's Q2 Market Share Up, AMD's Down," *InfoWorld*, July 31, 2002, <archive.infoworld.com/articles/hn/xml/02/07/31/020731hnstudy.xml>.
26. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002, pp. 5-2,5-5.
27. *IA-32 Intel Architecture Software Developer's Manual*, Vol. 3, *System Programmer's Guide*, 2002, pp. 2-8.
28. Bar, M., "Kernel Korner: The Linux Signals Handling Model," *Linux Journal*, May 2000, <www.linuxjournal.com/article.php?sid=3985>.
29. Bovet, D. and M. Cesati, *Understanding the Linux Kernel*, O'Reilly, 2001, p. 253.
30. Bar, M., "Kernel Korner: The Linux Signals Handling Model," *Linux Journal*, May 2000, <www.linuxjournal.com/article.php?sid=3985>.
31. Gentleman, W. M., "Message Passing Between Sequential Processes: The Reply Primitive and the Administrator Concept," *Software—Practice and Experience*, Vol. 11, 1981, pp. 435-466.
32. Schlichting, R. D. and F. B. Schneider, "Understanding and Using Asynchronous Message Passing Primitives," in *Proceedings of the Symposium on Principles of Distributed Computing*, August 18-20, 1982, Ottawa, Canada, ACM, New York, pp. 141-147.
33. Stankovic, J. A., "Software Communication Mechanisms: Procedure Calls versus Messages," *Computer*, Vol. 15, No. 4, April 1982.
34. Staustrop, X, "Message Passing Communication versus Procedure Call Communication," *Software—Practice and Experience*, Vol. 12, No. 3, March 1982, pp. 223-234.
35. Cheriton, D R., "An Experiment Using Registers for Fast Message-Based Interprocess Communications," *Operating Systems Review*, Vol. 18, No. 4, October 1984, pp. 12-20.
36. Olson, R., "Parallel Processing in a Message-Based Operating System," *IEEE Software*, Vol. 2, No. 4, July 1985, pp. 39-49.
37. Andrews, G. R., "Synchronizing Resources," *ACM Transactions on Programming Languages and Systems*, Vol. 3, No. 4, October 1981, pp. 405-430.
38. Andrews, G., and F. Schneider, "Concepts and Notations for Concurrent Programming," *ACM Computing Surveys*, Vol. 15, No. 1, March 1983, pp. 3-44.
39. Bovet, D., and M. Cesati, *Understanding the Linux Kernel*, O'Reilly, 2001, pp. 524-532.

40. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
41. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
42. *FreeBSD Handbook: Processes*, 2002, <www.freebsd.org/handbook/basics-processes.html>.
43. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
44. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
45. Ritchie, D., and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, July 1974, pp. 370-372.
46. *FreeBSD Handbook: Processes*, 2002, <www.freebsd.org/handbook/basics-processes.html>.
47. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
48. Lucent Technologies, "The Creation of the UNIX Operating System," 2002, <www.bell-labs.com/history/unix/>.
49. Organick, E., *The Multics System: An Examination of Its Structure*, Cambridge, MA: MIT Press, 1972.
50. Lucent Technologies, "The Creation of the UNIX Operating System," 2002, <www.bell-labs.com/history/unix/>.
51. Sun Microsystems, "Executive Bios: Bill Joy," <www.sun.com/aboutsun/media/ceo/mgt_joy.html>.
52. Calkins, B., "The History of Solaris," December 15, 2001, <unixed.com/Resources/history_of_solaris.pdf>.
53. Lucent Technologies, "The Creation of the UNIX Operating System," 2002, <www.bell-labs.com/history/unix/>.
54. Torvalds, L., "Linux History," July 31, 1992, <www.li.org/linuxhistory.php>.
55. Holland, N., "1-Introduction to OpenBSD," July 23, 2003, <www.openbsd.org/faq/faq1.html>.
56. Howard, J., "Daemon News: The BSD Family Tree," April 2001, <www.daemonnews.org/200104/bsd_family.html>.
57. Jorm, D., "An Overview of OpenBSD Security," August 8, 2000, <www.onlamp.com/pub/a/bsd/2000/08/08/OpenBSD.html>.
58. Security Electronics Magazine, "OpenBSD: Secure by Default," January 2002, <www.semweb.com/jan02/itsecurityjan.htm>.
59. Howard, J., "Daemon News: The BSD Family Tree," April 2001, <www.daemonnews.org/200104/bsd_family.html>.
60. The NetBSD Foundation, Inc., "About the NetBSD Project," July 17, 2003, <www.netbsd.org/Misc/about.html>.
61. Howard, J., "Daemon News: The BSD Family Tree," April 2001, <www.daemonnews.org/200104/bsd_family.html>.
62. Coelho, J., "comp.aix.unix Frequently Asked Questions (Part 1 of 5)," October 10, 2000, <www.faqs.org/faqs/aix-faq/part1/>.
63. IBM, "AIX Affinity with Linux: Technology Paper," <www-1.ibm.com/servers/aix/products/aixos/linux/affinity_linux.pdf>.
64. Springer, I., "comp.sys.hp.hpux FAQ," September 20, 2003, <www.faqs.org/faqs/hp/hpux-faq/>.
65. Hewlett-Packard Company, "HP-UX 11i Operating System," <www.hp.com/products1/unix/operating/>.
66. Hewlett-Packard Company, "Hewlett Packard Receives Top UNIX Ranking from D.H. Brown," May 30, 2002, <www.hp.com/hpinfo/newsroom/press/30may02b.htm>.
67. *FreeBSD Hypertext Man Pages*, 2002 <www.freebsd.org/cgi/man.cgi?query=execve&sektion=2>.
68. Ritchie, D., and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, July 1974, pp. 370-372.
69. Ritchie, D., and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, July 1974, pp. 370-372.
70. "Exit," *The Open Group Base Specifications, Issue 6, IEEE Std 1003.1*, 2003 Edition, <www.opengroup.org/onlinepubs/007904975/functions/Exit.html>.
71. *UNIXhelp for Users, Version 1.3.2: Nice*, <unixhelp.ed.ac.uk/CC1/man-cgi?nice>.
72. Thompson, K., "UNIX Implementation," *UNIX Programmer's Manual: 7th ed.*, Vol. 2b, January 1979, <cm.bell-labs.com/7thEdMan/bswv7.html>.
73. *The Design and Implementation of 4.4BSD Operating System: Interprocess Communication*, 2002, <www.freebsd.org/doc/en_US.ISO8859-1/books/design-44bsd/x659.html>.
74. Horning, J. J., and B. Randell, "Process Structuring," *ACM Computing Surveys*, Vol. 5, No. 1, March 1973, pp. 5-29.
75. Lampson, B. W., "A Scheduling Philosophy for Multiprocessing System," *Communications of the ACM*, Vol. 11, No. 5, 1968, pp. 347-360.
76. Peterson, J. L.; J. S. Quarterman; and A. Silbershatz, "4.2BSD and 4.3BSD as Examples of the UNIX System," *ACM Computing Surveys*, Vol. 17, No. 4, December 1985, p. 388.
77. Hennessy, J., and D. Patterson, *Computer Organization and Design*, San Francisco: Morgan Kaufmann Publishers, Inc., 1998.
78. Guillemont, M., "The Chorus Distributed Operating System: Design and Implementation," *Proceedings of the ACM International Symposium on Local Computer Networks*, Firenze, April 1982, pp. 207-223.
79. Accetta, M. J; R. V. Baron; W. Bolosky; D. B. Golub; R. F. Rashid; A. Tevanian; and M. W. Young, "Mach: A New Kernel Foundation for UNIX Development," *Proceedings of the Usenix Summer'86 Conference*, Atlanta, Georgia, June 1986, pp. 93-113.
80. Liedtke, Jochen, "Improving IPC by Kernel Design," *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*, December 5-8, 1993, pp. 175-188.
81. Liedtke, Jochen, "Toward Real Microkernels," *Communications of the ACM*, Vol. 39, No. 9, September 1996, pp. 70-77.
82. Bovet, D., and M. Cesati, *Understanding the Linux Kernel*, O'Reilly, 2001, p. 253.
83. Solomon, D., and M. Russinovich, *Inside Windows 2000*, 3d ed., Redmond: Microsoft Press, 2000.

*The spider's touch, how exquisitely fine!
Feels at each thread, and lives along the line.*

— Alexander Pope—

Yon cannot conceive the many without the one.

— Plato—

There is a time for many words, and there is also a time for sleep.

—Homer—

To be awake is to be alive.

— Henry David Thoreau —

Only a signal shown and a distant voice in the darkness.

—Henry Wadsworth Longfellow—

Chapter 4

Thread Concepts

Objectives

After reading this chapter, you should understand:

- *the motivation for creating threads.*
- *the similarities and differences between processes and threads.*
- *the various levels of support for threads.*
- *the life cycle of a thread.*
- *thread signaling and cancellation.*
- *the basics of POSIX, Linux, Windows XP and Java threads.*

Chapter Outline

4.1

Introduction

*Operating Systems Thinking:
Concurrency*

4.2

Definition of Thread

4.3

Motivation for Threads

*Operating Systems Thinking:
Parallelism*

4.4

Thread States: Life Cycle of a Thread

4.5

Thread Operations

4.6

Threading Models

4.6.1 User-Level Threads

4.6.2 Kernel-Level Threads

*Operating Systems Thinking:
Standards Conformance*

4.6.3 Combining User- and Kernel-level Threads

4.7

Thread Implementation Considerations

4.7.1 Thread Signal Delivery

Anecdote: Engineering

4.7.2 Thread Termination

4.8

FOSIX and Pthreads

*Anecdote: Standards and Conformance:
Plug-to-Plug Compatibility*

4.9

Linux Threads

*Operating Systems Thinking:
Scalability*

4.10

Windows XP Threads

4.11

Java Multithreading Case Study, Part I:

Introduction to Java Threads

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

4.1 Introduction

Early operating systems enabled computers to execute several programs concurrently, but the programming languages of the time did not enable programmers to specify concurrent activities (see the Operating Systems Thinking feature, Concurrency). Rather, these languages generally provided only a simple set of control structures that enabled programmers to specify a single thread of control. The types of concurrent operations that computers performed were generally implemented by operating system primitives available only to highly experienced systems programmers.

The Ada programming language, developed by the United States Department of Defense in the late 1970s and early 1980s, was one of the first languages to provide explicit concurrency primitives. Ada was made widely available to defense contractors building military command-and-control systems. However, Ada has not been widely adopted by universities and commercial industry.



Operating Systems Thinking

Concurrency

We will see numerous examples in the text of things that can happen concurrently. I/O operations can proceed concurrently with program execution, several processors can be executing concurrently, several users can be using a system concurrently. Several processes can be trying to access shared data concurrently. Several computers can be operating on the same network concurrently.

We study concurrency issues throughout the book. This and the preceding chapter discuss processes and threads—abstractions used by the operating system to manage concurrent activities. Chapters 5 and 6 are concerned

with concurrent programming and the subtle issues of enabling concurrent processes and threads to work together to solve common problems of coordination and data sharing. Chapter 7 deals with deadlock and indefinite postponement of concurrent processes and threads. Chapter 8 deals with scheduling a processor among concurrent processes and threads. Chapters 9 and 11 deal with memory organization and management issues among concurrent threads and processes. Chapter 15 deals with concurrent processes and threads on multi-processor systems. Chapter 16 deals with computer networking

and the fascinating protocols used to ensure that computers functioning concurrently on the same network do not "collide" with one another. Chapters 17 and 18 deal with the problems of building concurrent applications, pieces of which are distributed across a computer network. Chapter 19 deals with issues of security and protection among concurrent users operating on individual and networked computers. The case study chapters on Linux and Windows XP show real-world implementations of concurrency control.

In recent years, many general-purpose programming languages, including Java, C#, Visual C++ .NET, Visual Basic .NET and Python, have made concurrency primitives available to the applications programmer. The programmer specifies that applications contain "threads of execution," each thread designating a portion of a program that may execute concurrently with the other threads. This technology, called **multithreading**, gives the programmer powerful capabilities not directly available in languages such as C and C++, the languages on which Java and C# are based. C and C++ are called single-threaded languages. [Note: On many computer platforms, C and C++ programs can perform multithreading by using specific code libraries, but these are not part of the ANSI/ISO standard versions of these languages.] Operating system support for threads is essential to supporting languages that provide multithreading semantics.

Writing multithreaded programs can be tricky. Although the human mind can perform functions concurrently, people find it difficult to jump between parallel "trains of thought." To see why multithreaded applications can be difficult to program and understand, try the following experiment: Open three books to page 1 and try reading the books concurrently. Read a few words from the first book, then read a few words from the second book, then read a few words from the third book, then loop back and read the next few words from the first book, and so on. After this experiment, you will appreciate some of the key challenges of multithreading—switching between books, reading briefly, remembering your place in each book, moving the book you are reading closer so you can see it, pushing books you are not reading aside—and amid all this chaos, trying to comprehend the content of the books!

Self Review

1. The text mentions that multithreading capabilities are not directly available in languages such as C and C++. How do programmers still manage to write multithreaded code in these languages?
2. What key advantage would you get by running a multithreaded application on a multiprocessor system over running it on a uniprocessor system?

Ans: **1)** There are specific code libraries that help with multithreading. However, these libraries are not part of the ANSI/ISO C and C++ standards, so programs written using them are not as portable as "standard C and C++" programs. **2)** The multiple threads of the application that perform parallel tasks could run truly simultaneously on separate processors, speeding application execution.

4.2 Definition of Thread

Due to the broad support for multithreading in programming languages, virtually all recent operating systems provide at least some support for threads. A thread, sometimes called a **lightweight process (LWP)**, shares many attributes of a process. Threads are scheduled on a processor, and each thread can execute a set of instructions independent of other processes and threads. However, threads are not meant

to exist alone—they normally belong to traditional processes, sometimes referred to as **heavyweight processes (HWP)**. Threads within a process share many of the process's resources—most notably its address space and open files—to improve the efficiency with which they perform their tasks. The name "thread" refers to a single thread of instructions or thread of control; threads within a process can execute concurrently and cooperate to attain a common goal. On a multiprocessor system, multiple threads may be able to execute simultaneously.

Threads possess a subset of the resources contained in a process. Resources such as processor registers, the stack and other thread-specific data (TSD), such as signal masks (data that describes which signals a thread will not receive, discussed in Section 4.7.1, Thread Signal Delivery) are local to each thread, while the address space belongs to the process that contains the threads and is global to the threads (Fig. 4.1). Depending on the thread implementation for the particular platform, threads may be managed by the operating system or by the user application that creates them.

Although many operating systems support threads, the implementations vary considerably. Win32 threads,¹ C-threads² and POSIX threads³ are examples of threading libraries with disparate APIs. **Win32 threads** are used in the Microsoft 32-bit Windows operating systems; **C-threads** are created from a thread library in the Mach microkernel (on which Macintosh OS X is built) and are also supported by Solaris and Windows NT operating systems. The POSIX specification provides the **Pthreads** standard. The primary goal of Pthreads is to allow multithreaded programs to be portable across multiple operating system platforms. POSIX has been implemented in a variety of operating systems, including Solaris, Linux and Windows XP.

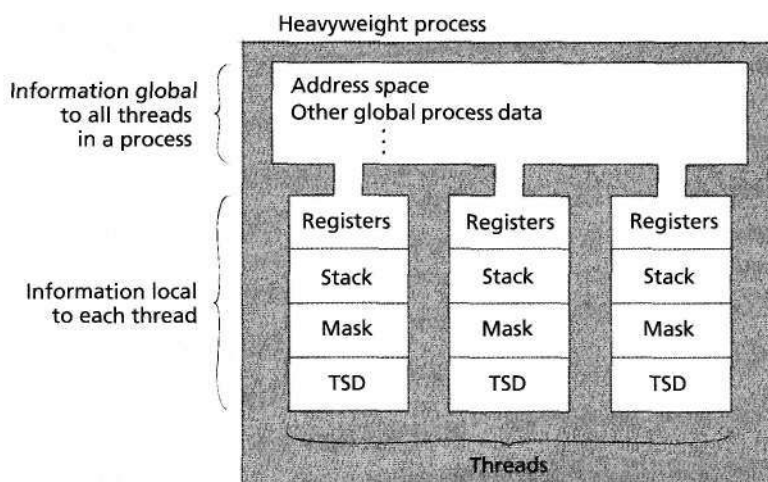


Figure 4.1 | Thread relationship to processes.

Self Review

1. Why are traditional processes called heavyweight processes?
2. Why is it difficult to write portable multithreaded applications?

Ans: 1) The primary distinction between relative "weights" of traditional processes and threads is how address spaces are allocated. When a process is created, it is allocated its own address space. When a thread is created, it shares the process's address space, so threads are "lighter weight" than processes. 2) There is no standard threading library that is implemented on all platforms.

4.3 Motivation for Threads

In the previous chapter, we introduced the notion of a process and described how computer systems benefit from improved efficiency and performance when multiple processes execute concurrently. When the process concept was introduced by the Multics project in the 1960s, computers typically contained a single processor and applications were relatively small.⁴ Processes of that era were designed to execute a single thread of control on one processor at a time. Subsequent trends in software and hardware design indicated that systems could benefit from multiple threads of execution per process. Some motivating factors for multithreading are:

- *Software design*—Due to modularity and compiler design, many of today's applications contain segments of code that can be executed independent of the rest of the application. Separating independent code segments into individual threads can improve application performance and can make inherently parallel tasks simpler to express in code (see the Operating Systems Thinking feature, Parallelism).
- *Performance*—A problem with single-threaded applications is independent activities cannot be scheduled to execute on multiple processors. In a multithreaded application, threads can share a processor (or set of processors), so that multiple tasks are performed in parallel. Concurrent parallel execution can significantly reduce the time required for a multithreaded application to complete its task, especially in multiprocessor systems, when compared to a single-threaded application that can execute on only one processor at a time and must perform its operations sequentially. Also, in multithreaded processes, ready threads can execute while others are blocked (i.e., while awaiting I/O completion).
- *Cooperation*—Many applications rely on independent components to communicate and synchronize activities. Before threads, these components executed as multiple "heavyweight" processes that established interprocess communication channels via the kernel.^{5, 6} Performance with a multiple-lightweight-thread approach is often much better than that with a multiple-heavyweight-process approach because a process's threads can communicate using their shared address space.

Multithreaded applications are ubiquitous in today's computer systems. A Web server is one environment in which threads can dramatically improve performance and interactivity. Web servers typically receive requests from remote applications for Web pages, images and other files. It is common for Web servers to service each request with a separate thread. The process that receives requests may contain one thread that listens for requests from the Internet. For each request received, a new thread is spawned that interprets the request, retrieves the specified Web page and transmits the Web page to the client (typically a Web browser). After the new thread is spawned, its parent can continue to listen for new requests. Because many Web servers are multiprocessor systems, several requests can be received and fulfilled concurrently by different threads, improving both throughput and response time. The overhead incurred by creating and destroying a thread to service each request is substantial. As a result, most of today's Web servers maintain a pool of threads that are assigned to service new requests as they arrive. These threads are not destroyed after servicing the request; rather, they are reassigned to the pool to be reassigned to incoming requests. We discuss thread pooling in greater detail in Section 4.6.3, Combining User- and Kernel-Level Threads.

Word processors use threads to enhance user productivity and improve interactivity. Each time the user types a character at the keyboard, the operating system receives a keyboard interrupt and issues a signal to the word processor. The word processor responds by storing the character in memory and displaying it on the



Operating Systems Thinking

Parallelism

One way to implement parallelism is to do so on the local machine with techniques like multiprogramming, multithreading, multiprocessing and massive parallelism. Computer hardware is built to be able to perform processing in parallel with input/output. Multiprocessors are built to have several processors working in parallel—massive parallelism carries that to the extreme with hun-

dreds, thousands or even more processors working in parallel.

Today, another kind of parallelism is becoming prominent, namely distributed computing over computer networks. We study distributed computing in Chapters 16 through 18 where we examine computer networking and the issues of building distributed operating systems. An operating system is primarily a

resource manager. For years, those resources were the hardware, software and data of the local computer system. Today, a distributed operating system must manage resources no matter where they reside, whether on the local computer system or on computer systems distributed across computer networks like the Internet.

screen. Because today's computers can execute hundreds of millions of processor instructions between successive keystrokes, word processors can execute several other threads between keyboard interrupts. For example, many of today's word processors detect misspelled words as they are typed and periodically save a copy of the document to disk to prevent loss of data. Each feature is implemented with a separate thread—as a result, the word processor can respond to keyboard interrupts even if one or more of its threads are blocked due to an I/O operation (e.g., saving a copy of the file to disk).

Self Review

1. How does improved software design help to make multithreaded applications execute faster?
2. Why is it typically more efficient for threads of the same process to communicate than it is for separate processes to communicate?

Ans: 1) Many applications contain segments of code that can execute independently of one another. When these segments of code are assigned to separate threads, they can, for example, execute on multiple processors simultaneously. 2) Threads of the same process can communicate via their shared address space and do not have to rely on IPC mechanisms that invoke the kernel.

4.4 Thread States: Life Cycle of a Thread

As discussed in Section 3.2, Process States: Life Cycle of a Process, each process can be viewed as transitioning between a series of discrete process states. In this model, each process contains a single thread of control; therefore, we could have also stated that each thread of control moves through a series of discrete states. When processes contain multiple threads of control, we can view each thread as transitioning between a series of discrete **thread states**. Thus, much of the discussion of process states and state transitions in Section 3.2, applies to thread states and state transitions.

For example, consider the following set of states, based largely on the Java thread implementation (Fig. 4.2).⁷ In Java, a new thread begins its life cycle in the **born** state. It remains in the *born* state until the program starts the thread, which places the thread in the **ready** state (sometimes called the **runnable** state). In other operating systems, a thread is started upon creation, eliminating the *born* state. The highest-priority *ready* thread enters the **running** state (i.e., begins executing) when it obtains a processor.

A *running* thread enters the **dead** state when it completes its task or otherwise terminates. Some threading libraries allow a thread to terminate another thread, which forces the latter into the *dead* state. Once a thread enters the *dead* state, its resources are released and it is removed from the system.

A thread enters the **blocked** state when it must wait for the completion of an I/O request (e.g., reading data from disk). A *blocked* thread is not dispatched to a processor until its I/O request has been completed. At that point, the thread returns to the *ready* state, so that it can resume execution when a processor becomes available.

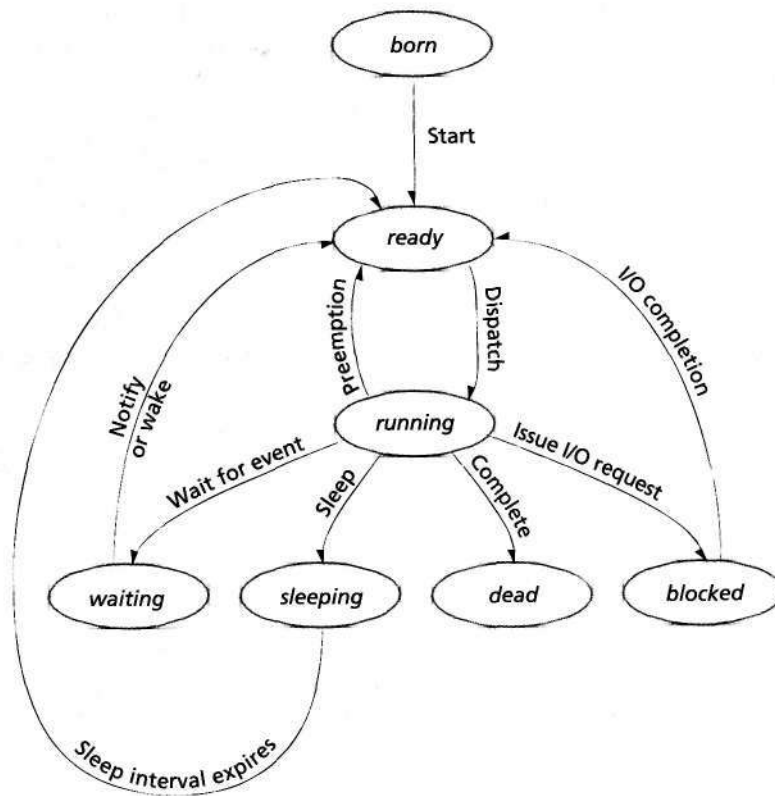


Figure 4.2 | Thread life cycle.

When a thread must wait for an event (e.g., mouse movement or a signal from another thread), it can enter the **waiting** state. Once in this state, it returns to the **ready** state when another thread **notifies** it (the term **awakens** it is also used). When a **waiting** thread receives a notify event, the thread transitions from **waiting** to **ready**.

A **running** thread can enter the **sleeping** state for a specified period of time (called a **sleep interval**). A **sleeping** thread returns to the **ready** state when its designated sleep interval expires. **Sleeping** threads cannot use a processor, even if one is available. Threads sleep when they momentarily do not have work to perform. For example, a word processor may contain a thread that periodically writes a copy of the current document to disk for recovery purposes. If the thread did not sleep between successive backups, it would require a loop in which it continually tests if it should write a copy of the document to disk. This loop would consume processor time without performing productive work, reducing system performance. In this case, it is more efficient for the thread to specify a sleep interval (equal to the period between successive backups) and enter the **sleeping** state. The **sleeping**

thread is returned to the *ready* state when its sleep interval expires, at which point it writes a copy of the document to disk and reenters the *sleeping* state.

Self Review

1. How can a thread enter the *dead* state?
2. How are the waiting, blocked and sleeping states similar? How are they different?

Ans: 1) A thread enters the dead state when it completes its task or when another thread terminates it. **2)** These states are similar in that the threads in them could not use a processor, even if one were available. A *blocked* thread cannot be dispatched because it is waiting on an I/O operation that it requested. In this case, the operating system is responsible for ensuring that it eventually unblocks the thread. A *waiting* thread cannot be dispatched until it receives an event from hardware or software that the operating system does not initiate (e.g., a key press or a signal from another thread). In this case, the operating system cannot control whether or when a *waiting* thread will eventually be awakened. A *sleeping* thread cannot execute because it has explicitly notified the system that it should not execute until its sleep interval expires.

4.5 Thread Operations

Threads and processes have many operations in common such as

- create
- exit (i.e., terminate)
- suspend
- resume
- sleep
- wake.

In many ways, thread creation is similar to process creation. When a process spawns a thread, the threading library initializes thread-specific data structures that store information such as register contents, the program counter and a thread ID. Unlike process creation, thread creation does not require the operating system to initialize resources that are shared between the parent process and its threads (e.g., the address space). Many operating systems require fewer instructions to share resources than they do to initialize them, so in these systems, thread creation is faster than process creation.⁸ Likewise, thread termination is often faster than process termination. The reduced overhead due to thread creation and termination encourages software developers to implement parallel tasks using multiple threads instead of multiple processes, when feasible.

Some thread operations do not correspond precisely to process operations. They include:

- **cancel**—A thread or process can cause a thread to terminate prematurely by cancelling it. Unlike process termination, thread cancellation is not guaranteed to terminate the thread. This is because threads may disable, or

mask, signals; if a thread masks the cancellation signal, it will not receive the signal until the thread reenables the cancellation signal.⁹ However, a thread cannot mask an abort signal.

- **join**—In some thread implementations (e.g., Windows XP), when a process is initialized, it creates a **primary thread**. The primary thread acts as any other thread, except that if it returns, the process terminates. To prevent a process from terminating before all its threads complete execution, the primary thread typically sleeps until each thread it creates has completed execution. In this case, the primary thread is said to join each of the threads it creates. When a thread joins another thread, the former does not execute until the latter terminates.¹⁰

Although most thread implementations support the operations discussed in this section, other thread operations are specific to particular thread libraries. In the sections that follow, we introduce several popular thread implementations and discuss issues the designer must address when creating a threading library.

Self Review

1. How does the cancellation signal differ from the abort signal, discussed in Section 3.5.1, Signals?
2. Why does thread creation typically require fewer processor cycles than process creation?

Ans: **1)** When a thread receives an abort, it is terminated immediately (i.e., threads cannot mask abort signals). When a thread is cancelled, it may continue to exist until it unmask the cancellation signal. **2)** Unlike process creation, thread creation requires that the operating system initialize only resources that are local to the thread of control. Global resources (e.g., the process's address space) can be shared using pointers, which requires significantly fewer processor cycles than initialization.

4.6 Threading Models

Thread implementations vary among operating systems, but almost all operating systems support one of three primary threading models. This section examines the three most popular threading models: user-level threads, kernel-level threads, and a combination of the two.

4.6.1 User-level Threads

Early operating systems supported processes that contained only a single execution context.¹¹ As a result, each multithreaded process was responsible for maintaining thread state information, scheduling threads and providing thread-synchronization primitives. These **user-level threads** perform threading operations in user space, meaning that threads are created by runtime libraries that cannot execute privileged instructions or access kernel primitives directly.¹² User-level threads are transparent to the operating system—it treats each multithreaded process as a single execution context. This means that the operating system dispatches the multithreaded process as a unit, as opposed to dispatching each individual thread. For

this reason, user-level thread implementations are also called **many-to-one** thread mappings, because the operating system maps all threads in a multithreaded process to a single execution context (Fig. 4.3).

When a process employs user-level threads, user-level libraries perform scheduling and dispatching operations on the process's threads because the operating system is unaware that the process contains multiple threads. The multithreaded process continues to execute until its quantum expires or until it is otherwise preempted by the kernel.¹³

There are several benefits to implementing threads in user space instead of in kernel space. User-level threads do not require that the operating system support threads. Therefore, user-level threads are more portable because they do not rely on a particular operating system's threading API. Another advantage is that, because the threading library, not the operating system, controls how threads are scheduled, application developers can tune the threading library's scheduling algorithm to meet the needs of specific applications.¹⁴

Also, user-level threads do not invoke the kernel for scheduling decisions or synchronization procedures. Recall from Section 3.4.1, Interrupt Processing, that a system loses a number of processor cycles to overhead when an interrupt, such as a system call, occurs. Therefore, user-level multithreaded processes that frequently perform threading operations (e.g., scheduling and synchronization) benefit from low overhead relative to threads that rely on the kernel for such operations.¹⁵

User-level thread performance varies depending on the system and on process behavior. Many of the deficiencies of user-level threads relate to the fact that the kernel views a multithreaded process as a single thread of control. For instance, user-level threads do not scale well to multiprocessor systems, because the kernel cannot simultaneously dispatch a process's threads to multiple processors, so user-level

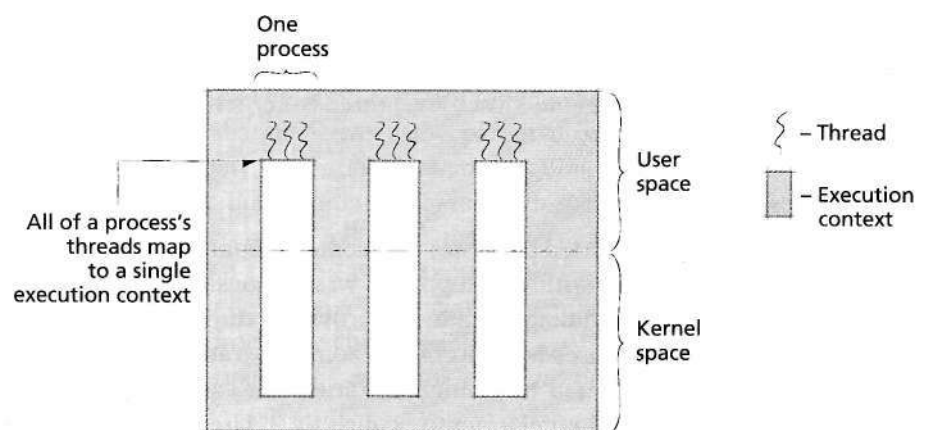


Figure 4.3 | User-level threads.

threads can result in suboptimal performance in multiprocessor systems.¹⁶ In a many-to-one thread mapping, the entire process blocks when any of its threads requests a blocking I/O operation, because the entire multithreaded process is the only thread of control that the operating system recognizes. Even if the multithreaded process contains threads in the *ready* state, none of its threads can execute until the *blocked* thread becomes *ready*. This can slow progress to a crawl for multithreaded processes that block frequently. After a user-level thread blocks, the kernel will dispatch mother process, which may contain threads of a lower priority than the *ready* threads contained in the *blocked* process. Therefore, user-level threads also do not support systemwide scheduling priority, which could be particularly detrimental to real-time multithreaded processes.¹⁷ Note that some threading libraries translate blocking system calls into nonblocking system calls to address this problem.¹⁸

Self Review

1. Explain why user-level thread implementations promote portability.
2. In many-to-one thread mappings, why does the operating system block the entire multithreaded process when a single thread blocks?

Ans: 1) User-level threads present an API to applications that is independent of the operating system's API. 2) To the operating system, the entire multithreaded process is a single thread of control. Therefore, when the operating system receives a blocking I/O request, it blocks the entire process.

4.6.2 Kernel-level Threads

Kernel-level threads attempt to address the limitations of user-level threads by mapping each thread to its own execution context. As a result, kernel-level threads are often described as a **one-to-one thread mapping** (Fig. 4.4). Such mappings require the operating system to provide each user thread with a kernel thread that

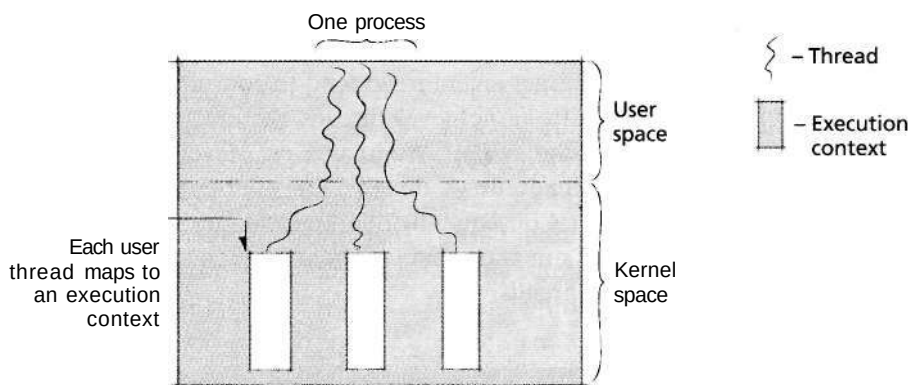


Figure 4.4 | Kernel-level threads.

the operating system can dispatch. Kernel threads differ from heavyweight processes because they share their process's address space. Each kernel thread also stores thread-specific data, such as register contents and a thread identifier, for each thread in the system. When a user process requests a kernel-level thread using system calls defined by the operating system's API, the operating system creates a kernel thread that executes the user thread's instructions.

There are several benefits to a one-to-one mapping. The kernel can dispatch a process's threads to several processors at once, which can improve performance for applications designed for concurrent execution.¹⁹ Also, the kernel can manage each thread individually, meaning that the operating system can dispatch a process's *ready* threads even if one of its threads is *blocked*. Therefore, applications that perform blocking I/O can execute other threads while waiting for the I/O operation to complete. This can improve interactivity for applications that must respond to user input and can improve performance, in general, as long as the application can benefit from concurrent execution.

Kernel-level threads enable the operating system's dispatcher to recognize each user thread individually. If the operating system implements a priority-based scheduling algorithm, a process that uses kernel-level threads can adjust the level of service each thread receives from the operating system by assigning scheduling priorities to each of its threads.²⁰ For example, a process can improve its interactivity by assigning a high priority to a thread that responds to user requests and lower priorities to its other threads.

Kernel-level threads are not always the optimal solution for multithreaded applications. Kernel-level thread implementations tend to be less efficient than user-level thread implementations because scheduling and synchronization operations invoke the kernel, which increases overhead. Also, software that employs kernel-level threads is often less portable than software that employs user-level threads—the application programmer that uses kernel-level threads must modify the program to use the thread API for each operating system on which it runs.²¹ Operating systems that conform to standard interfaces such as POSIX reduce this problem (see the Operating Systems Thinking feature, Standards Conformance). Another disadvantage is that kernel-level threads tend to consume more resources than user-level threads.²² Finally, kernel-level threads require that the operating system manage all threads in the system. Whereas a user-level library might be required to manage tens or hundreds of threads, the operating system may be required to manage thousands. Consequently, the application developer must be sure that the operating system's memory management and scheduling subsystems scale well to large numbers of threads.

Self Review

1. In what scenarios are kernel-level threads more efficient than user-level threads?
2. Why is application software written for kernel-level threads less portable than software written for user-level threads?

Ans: 1) If an application contains threads that block or that can perform their instructions in parallel, kernel-level threads are more efficient than user-level threads. 2) Application software written using kernel-level threads depends on a particular operating system's thread API.

4.6.3 Combining User- and Kernel-level Threads

Some operating systems, such as Solaris and Windows XP, have attempted to bridge the gap between many-to-one and one-to-one mappings by creating a hybrid implementation of threads. The combination of the user- and kernel-level thread implementation is known as the **many-to-many** thread mapping (Fig. 4.5).²³ As its name suggests, this implementation maps many user-level threads to a set of kernel threads. Some refer to this technique as **m-to-n thread mapping**, because the number of user threads and the number of kernel threads need not be equal.²⁴

One-to-one thread mappings require that the operating system allocate data structures that represent kernel threads. As a result, the amount of memory consumed by kernel thread data structures can become significant as the number of threads in the system increases. Many-to-many thread mappings reduce this overhead by implementing **thread pooling**. This technique allows an application to specify the number of kernel-level threads it requires. For example, in Fig. 4.5, process P_1 has requested three kernel-level threads. Note that the threads T_1 and T_2 are mapped to a single kernel-level thread—a many-to-one mapping. This requires that



Operating Systems Thinking

Standard Conformance

Imagine trying to access the Internet without the standardized network protocols. Or imagine trying to buy a lamp if wall plugs were not standardized. Even more fundamentally, imagine what driving would be like if cities were to assign different meanings to red and green lights than the current standards of "stop" and "go." There are several major national and international organizations that promote computer-industry standards, such as POSIX, ANSI (the American National Stan-

dards Institute) in the United States, ISO (the International Organization for Standardization), OMG (the Object Management Group), W3C (the World Wide Web consortium) and many more.

An operating systems designer must be aware of the many standards to which a system must conform. These standards often are not static; rather, they evolve with the changing needs of the world community of computer and communications users.

There are drawbacks to standards as well—they can slow or even stifle innovation. They force operating systems development organizations (and others) to spend significant time and money conforming to a vast array of standards, many of which are obscure or even outdated. We refer to standards and standards organizations throughout the book and carefully consider the meaning and impact of standards conformance.

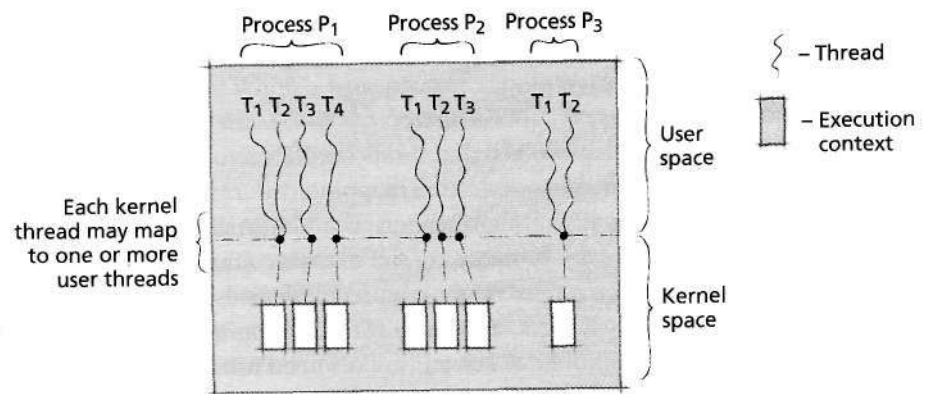


Figure 4.5 | Hybrid threading model.

ffif&rfct threading model.

the application maintain state information for each of its threads. Application developers are encouraged to use many-to-one mappings for threads that exhibit a low degree of parallelism (i.e., cannot benefit from simultaneous execution). Process P₁'s other threads (T₃ and T₄) are each mapped to a kernel-level thread. These threads are managed by the operating system, as discussed in the previous section.

Thread pooling can significantly reduce the number of costly thread creation and destruction operations. For example, Web and database systems often create a new thread to respond to each incoming request for service. Thread pooling allows kernel threads to remain in the system after a user thread dies. The kernel thread can then be allocated to a new user thread that is created at a later time. This improves system response times in environments such as Web servers because requests can be assigned to threads that already exist in the pool. These persistent kernel threads are called **worker threads** because they typically perform several different functions, depending on the threads that are assigned to them.²⁵

An advantage of a many-to-one thread mapping is that applications can improve performance by customizing the threading library's scheduling algorithm. However, as discussed in Section 4.6.1, User-Level Threads, if a single user-level thread blocks, the operating system blocks the entire multithreaded process. Another limitation to a many-to-one thread mapping is that a process's threads cannot simultaneously execute on multiple processors. Scheduler activations attempt to address these limitations to user-level threads. A **scheduler activation** is a kernel thread that can notify a user-level threading library of events (e.g., a thread has blocked or a processor is available). This type of kernel thread is called a "scheduler activation," because the user-level threading library can perform thread-scheduling operations when "activated" by an event notification, sometimes called an upcall.

When a multithreaded process is created, the operating system creates a scheduler activation that executes the process's user-level threading-library initialization code, which creates threads and requests additional processors for its

threads, if necessary. The operating system creates an additional scheduler activation for each processor allocated to a process, enabling the user-level library to assign different threads to execute simultaneously on multiple processors.

When a user-level thread blocks, the operating system saves the state of the thread to its scheduler activation and creates a new scheduler activation to notify the user-level library that one of its threads has blocked. The user-level threading library can then save the state of the *blocked* thread from its scheduler activation and assign a different thread to that scheduler activation. This mechanism prevents the entire multithreaded process from being blocked when one of its threads blocks.²⁶

The primary limitation of the many-to-many threading model is that it complicates operating system design, and there is no standard way in which to implement it.²⁷ For example, Solaris 2.2's many-to-many threading model allowed user applications to specify the number of kernel threads assigned to each process; Solaris 2.6 introduced scheduler activations. Interestingly, Solaris 8 abandoned its predecessors' many-to-many thread mapping in favor of a simpler and scalable one-to-one thread mapping scheme.²⁸ Windows XP, which does not support scheduler activations, dynamically adjusts the number of worker threads in its thread pools in response to system load.^{29–30}

Self Review

1. Why is it inefficient for an application to specify a thread pool size that is larger than the maximum number of *ready* user threads at any point during the application's execution?
2. How do scheduler activations improve performance in a many-to-many thread mapping?

Ans: 1) Each worker thread in the thread pool consumes system resources such as memory. If worker threads outnumber *ready* user threads, the system incurs overhead due to unnecessary thread creation and inefficient memory allocation. 2) Scheduler activations allow the application to indicate how threads should be scheduled to maximize throughput.

4.7 Thread Implementation Considerations

In this section, we discuss differences among thread implementations relating to thread signal delivery and thread cancellation. These differences highlight fundamental issues related to thread operations and thread management.

4.7.1 Thread Signal Delivery

Signals interrupt process execution, as do hardware interrupts, but signals are generated by software—either by the operating system or by user processes. Signals became a standard interprocess communication mechanism after appearing in the UNIX operating system. The original UNIX operating system did not support threads, so signals were designed for use with processes.³¹

As discussed in Section 3.5.1, Signals, when the operating system delivers a signal to a process, the process pauses execution and invokes a **signal handler** to

respond to the signal. When the signal handler completes, the process resumes execution (assuming the process has not exited).³²

There are two types of signals: synchronous and asynchronous. A **synchronous signal** occurs as the direct result of an instruction executed by the process or thread. For example, if a process or thread performs an illegal memory operation, the operating system sends the process a synchronous signal indicating the exception. An **asynchronous signal** occurs due to an event unrelated to the current instruction; such signals must specify a process ID to indicate the signal recipient. Asynchronous signals are commonly used to notify a process of I/O completion, suspend a process, continue a process or indicate that a process should terminate (see Section 20.10.1, Signals, for a list of POSIX signals).

When every process in the system contains a single thread of control, signal delivery is straightforward. If the signal is synchronous (e.g., an illegal memory operation), it is delivered to the process currently executing on the processor that initiated the signal (by generating an interrupt). If the signal is asynchronous, the operating system can deliver it to the target process if it is currently *running*, or the operating system can add the signal to a queue of **pending signals** to be delivered when the recipient process enters the *running* state.

Now consider the case of a multithreaded process. If the signal is synchronous, it is reasonable to deliver it to the thread currently executing on the processor that initiated the signal (by generating an interrupt). However, if the signal is asynchronous, the operating system must be able to identify the signal's recipient. One solution is to require that the sender specify a thread ID. However, if the process employs a user-level thread library, the operating system cannot determine which thread should receive the signal.

Alternatively, the operating system can implement signals such that the sender specifies a process ID. In this case, the operating system must decide whether to deliver the signal to all threads, several threads or one thread in the process. This may seem strange, but in fact this is the signal model that UNIX systems and the POSIX specification employ, and the aim is to provide compatibility with applications originally written for the UNIX operating system (see the Anecdote, Engineering).

According to the POSIX specification, processes send signals by specifying a process identifier, not a thread identifier. To solve the thread signal delivery problem, POSIX uses signal masking. A **signal mask** allows a thread to disable signals of a particular type, so that it does not receive signals of that type. A thread can thereby mask all signals except those that it wishes to receive (Fig. 4.6). In this approach, when the operating system receives a signal for a process, it delivers that signal to all threads in the process that are not masking signals of that type. Depending on the signal type and the default action (see Section 3.5.1, Signals), signals may be queued for delivery after the thread unmask the signal, or the signal may simply be dropped. For example, in Fig. 4.6, each shape represents a signal of a different type (e.g., suspend, resume, terminate). In this case, the operating system

attempts to deliver the triangle signal to a multithreaded process. Note that both threads 1 and 3 are masking the triangle signal. Thread 2 is not masking it; therefore, the operating system delivers the triangle signal to thread 2, which then invokes the process's corresponding signal handler.

Signal masking allows a process to divide signal handling among different threads. For example, a word processor may contain a thread that masks all signals except keyboard events. The sole purpose of this thread would be to record user keystrokes. Signal masking also enables the operating system to control which thread receives a signal.

When implementing a POSIX signal delivery mechanism, the operating system must be able to locate a signal mask for each user thread. A one-to-one thread mapping simplifies this problem because the operating system can attach a signal mask to each kernel thread, which corresponds to exactly one user thread. However, if the system employs a many-to-many model, signal masking can become complex. Consider the case where an asynchronous signal is generated for a process and the only thread that does not mask that signal is not currently *running*. In this case, the operating system can choose to add the signal to a list of pending signals or to drop the signal. As a general rule, because signals are often used to notify processes and threads of important events, the operating system should not drop signals. [Note: The POSIX specification dictates that the operating system may drop a signal if all threads in the process have masked it and the corresponding signal-handler action is to ignore the signal.³³]

One way to implement pending signals for a many-to-many threading model is to create a kernel thread for each multithreaded process that monitors and delivers its asynchronous signals. The Solaris 7 operating system employed a thread called the Asynchronous Signal Lightweight Process (ASLWP) that monitored sig-



Anecdote

The field of operating systems often seems more like engineering than science. A friend once told me his definition of engi-

neering: "If it's scratched paint it; if it won't fit, smash it and it will fit." This explanation is certainly crude, but it points to the

resourcefulness of operating systems designers.

Lesson to operating systems designers: Occasionally, when you dig deep into an operating system, you'll see examples of this kind of painting and smashing.

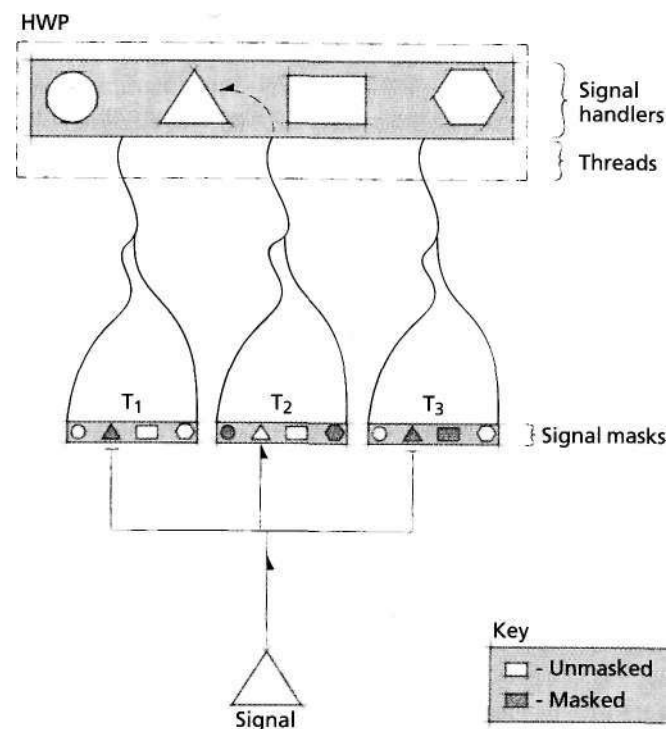


Figure 4.6 | Signal masking.

nals and managed pending signals so that they would be delivered to the appropriate thread, even if that thread was not *running* at the time of the signal.³⁴

If a multithreaded process employs a user-level library, the operating system simply delivers all signals to the process because it cannot distinguish individual threads. The user-level library registers signal handlers with the operating system, which are then executed upon receipt of a signal. The process's user-level thread library can then deliver the signal to any of its threads that do not mask it.³⁵⁻³⁶

Self Review

1. Why is synchronous signal delivery simpler than asynchronous signal delivery?
2. Explain how the ASLWP solves the signal-handling problem in a many-to-many threading model.

Ans: 1) Unlike an asynchronous signal, a synchronous signal is generated due to a process or thread that is currently executing on a processor. The operating system can easily identify the recipient of the signal by determining which process or thread is currently running on the processor that generated the interrupt corresponding to the signal. 2) An operating system can create a thread that stores each asynchronous signal until its recipient enters the *running* state (at which point the signal is delivered).

4.7.2 Thread Termination

When a thread terminates by completing execution normally (e.g., by an `exit` call to a threading library or by exiting the method that contains the thread's code), the operating system can immediately remove the thread from the system. Threads can also terminate prematurely, due to either an exception (such as an illegal memory reference) or a cancellation signal from a process or thread. Because threads cooperate by means such as modifying shared data, an application may produce subtle erroneous results when one of its threads unexpectedly terminates. Consequently, threading libraries must carefully determine how and when to remove the thread from the system. A thread can choose to disable cancellation by masking the cancellation signal. Typically it will do so only while performing a task that should not be interrupted before termination, such as completing a modification to a shared variable.³⁷

Self Review

1. Name three ways a thread can terminate.
2. Why should a thread be allowed to disable its cancellation signal?

Ans: 1) A thread can terminate by completing execution, raising a fatal exception or receiving a cancellation signal. **2)** A thread that modifies a value in its process's shared address space may leave data in an inconsistent state if terminated prematurely.

4.8 POSIX and Pthreads

POSIX (Portable Operating Systems Interface for Computing Environments) is a set of standards for operating system interfaces, published by the IEEE's Portable Application Standards Committee (PASC), that are largely based on UNIX System V.³⁸ The POSIX specification defines a standard interface between threads and their threading library (see the Anecdote, Standards and Conformance: Plug-to-Plug Compatibility). Threads that use the POSIX threading API are called **Pthreads** (sometimes referred to as POSIX threads or POSIX 1003.1c threads).³⁹ The POSIX specification is not concerned with the details of the implementation of the threading interface—Pthreads can be implemented in the kernel or by user-level libraries.

POSIX states that the processor registers, the stack and the signal mask are maintained individually for each thread, and any other resource information must be globally accessible to all threads in the process.⁴⁰ POSIX also defines a signal model to address many of the concerns discussed in Section 4.7, Thread Implementation Considerations. According to POSIX, when a thread generates a synchronous signal due to an exception such as an illegal memory operation, the signal is delivered only to that thread. If the signal is not specific to a thread, such as a signal to kill a process, then the threading library delivers that signal to a thread that does not mask it. If multiple threads leave the kill signal unmasked, that signal is delivered to one of those threads. More importantly, one may not use the kill signal to terminate a particular thread—if a thread acts upon a kill signal, the entire process, including all of its

threads, will terminate. This example demonstrates another important property of the POSIX signal model: although signal masks are stored individually in each thread, signal handlers are global to all of a process's threads.^{41,42}

To terminate a particular thread, POSIX provides a cancellation operation that specifies a target thread, the result of which depends on the target thread's cancellation mode. If the target thread chooses **asynchronous cancellation**, the thread can be terminated at any point during its execution. If the thread **defers cancellation**, it will not be cancelled until it explicitly checks for a cancellation request. Deferred cancellation allows a thread to complete a series of operations before being abruptly terminated. A thread may also **disable cancellation**, meaning that it is not notified that a cancellation operation has been requested.⁴³

In addition to the common operations discussed in Section 4.5, Thread Operations, the POSIX specification provides functions that support more advanced operations. It allows programs to specify various levels of parallelism and implement a variety of scheduling policies, including user-defined algorithms and real-time scheduling. The specification also addresses synchronization using locks, semaphores and condition variables (see Chapter 5, Asynchronous Concurrent Execution).^{44, 45, 46}

Few of today's most popular operating systems provide complete native Pthreads implementations, i.e., in the kernel. However, POSIX threading libraries exist to provide a wide range of support for various operating systems. For example, although Linux does not conform to POSIX by default, the Native POSIX Thread



Anecdote

Standards and Conformance: Plug-to-Plug Compatibility

The story goes that a manufacturer of tape drives in the 1960s wanted to produce tape drives that were equivalent in capabilities to those produced by a major computer vendor. Their business strategy was to advertise that their drives were "plug-to-plug compatible" with those of the major vendor and to sell the

drives at a lower price. The drives were built to conform to the major vendor's specifications. The problem was that when they tried to attach these tape drives to the computer systems of the major vendor, the drives would not work. They retested their drives and were puzzled by the fact that their drives were in perfect con-

formance with the specifications. Why didn't the drives work? Upon careful research, they discovered that the specification had bugs. They corrected these bugs and joked that maybe they should advertise their drives as being "bug-to-bug compatible" with those of the major vendor.

Lesson to operating systems designers: We live in a world of abundant and evolving standards. Achieving standards conformance is crucial to the success of modern operating systems. But standards are subject to human error just as operating systems are.

Library (NPTL) project aims to provide a conforming POSIX threading library that employs kernel-level threads in Linux.⁴⁷ Similarly, the interface to the Microsoft Windows line of operating systems (namely the Win32 API) does not conform to POSIX, but users can install a POSIX subsystem. [Note: Sun Microsystems's Solaris 9 operating system provides two threading libraries: a Pthreads library that conforms to POSIX and a legacy Solaris threads library (called UI threads). There is little difference between Pthreads and Solaris threads—Solaris has been designed so that calls to Pthread and Solaris thread functions from within the same application are valid.⁴⁸

Self Review

1. What is the primary reason to create standard thread interfaces such as Pthreads?
2. Which threading model does the POSIX standard require?

Ans: 1) Standard threading interfaces allow applications to be portable, which reduces software development time for applications that must operate on multiple platforms. 2) The POSIX standard does not require a specific implementation. Therefore, the threads can be implemented as user-level, kernel-level or hybrid threads.

4.9 Linux Threads

Support for threads in the Linux operating system was introduced as user-level threads in version 1.0.9 and as kernel-level threads in version 1.3.56.⁴⁹ Although Linux supports threads, it is important to note that many Linux kernel subsystems do not distinguish between threads and processes. In fact, Linux allocates the same type of process descriptor to processes and threads, both of which are called tasks. Linux uses the UNIX-based system call `fork` to spawn child tasks. Linux responds to the `fork` system call by creating a new task that contains a copy of all of its parent's resources (e.g., address space, register contents, stack).

To enable threading, Linux provides a modified version of the `fork` system call named `clone`. Similar to `fork`, `clone` creates a copy of the calling task—in the process hierarchy, the copy becomes the child of the task that issued the `clone` system call. Unlike `fork`, `clone` accepts arguments that specify which resources to share with the child process. At the highest level of resource sharing, tasks created by `clone` correspond to threads discussed in Section 4.2, Definition of Thread.

As of version 2.6 of the kernel, Linux provides a one-to-one thread mapping that supports an arbitrary number of threads in the system. All tasks are managed by the same scheduler, meaning that processes and threads with equal priority receive the same level of service. The scheduler has been designed to scale well to a large number of processes and threads. The combination of a one-to-one mapping and an efficient scheduling algorithm provides Linux with a highly scalable thread implementation (see the Operating Systems Thinking feature, Scalability). Although Linux does not support POSIX threads by default, it is distributed with a POSIX threading library. In the 2.4 kernel, a threading library called `LinuxThreads` provided POSIX functionality but did not entirely conform to the POSIX specifica-

tion. A more recent project, Native POSIX Thread Library (NPTL), has achieved nearly complete POSIX conformance and is likely to become the default threading library for the 2.6 kernel.⁵⁰

Each task in the process table stores information about the current task state (e.g., *running*, *stopped*, *dead*). A task in the *running* state may be dispatched to a processor (Fig. 4.7). A task enters the *sleeping* state when it sleeps, blocks or otherwise cannot execute on a processor. It enters the *stopped* state when it receives a stop (i.e., suspend) signal. The *zombie* state indicates that a task has been terminated but has not yet been removed from the system. For example, if a task contains several threads, it will enter the *zombie* state while notifying its threads that it received a termination signal. A task in the *dead* state may be removed from the system. These states are further discussed in Section 20.5.1, Process and Thread Organization.

Self Review

1. Explain the difference between the fork and clone system calls in Linux.
2. What is the difference between the *zombie* state and the *dead* state?

Ans: 1) When a task issues a fork system call, it spawns a child task and allocates it to a copy of its parent's resources. When a task issues a clone system call, the task specifies which resources it shares with the task it spawns. Tasks created using the clone system call are analogous to threads. 2) A task in the *zombie* state is not removed from the system so other threads can be notified of its termination. A task in the *dead* state may be immediately removed from the system.



Operating Systems Thinking

Scalability

Users' computing needs tend to increase with time. Operating systems need to be scalable, i.e., they need to be able to adjust dynamically as more hardware and software capabilities are added to a system. A multiprocessor operating system, for example, should

scale smoothly from managing a two-processor configuration to managing a four-processor configuration. We will see that most systems today employ a device-driver architecture that makes it easy to add new types of devices, even ones that did not exist when

the operating system was implemented. Throughout the book, we discuss techniques for making operating systems scalable. We conclude with discussions of scalability in Linux and Windows XP.

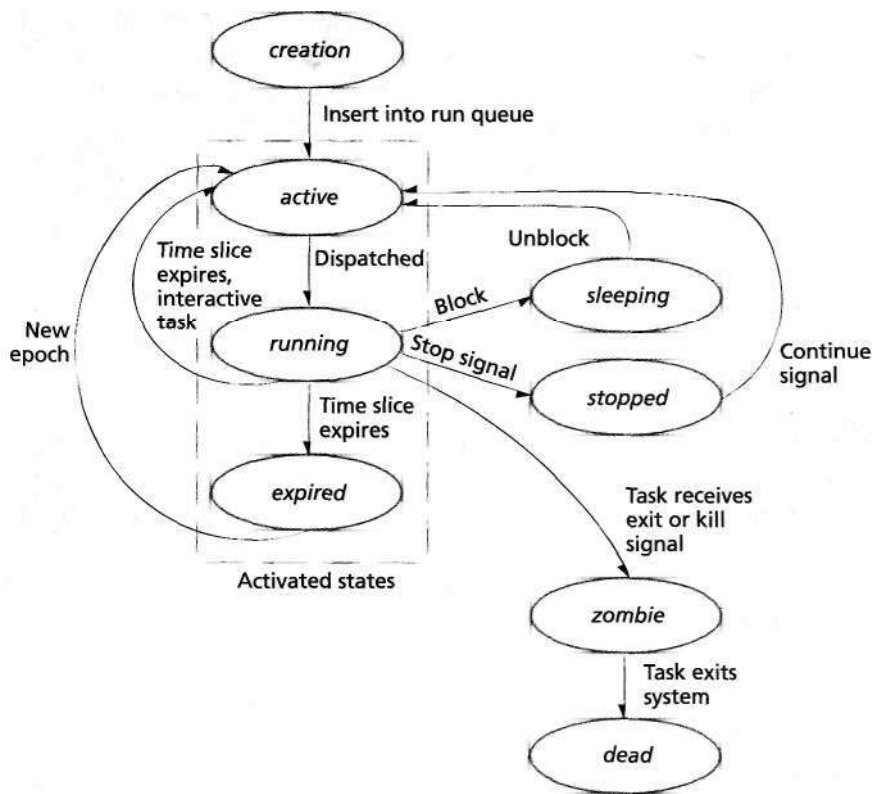


Figure 4.7 | Linux task state-transition diagram.

4.10 Windows XP Threads

In Windows XP, a process consists of program code, an execution context, resources (e.g., open files) and one or more associated threads. The execution context includes such items as the process's virtual address space and various attributes (e.g., security attributes). Threads are the actual unit of execution; threads execute a piece of a process's code in the process's context, using the process's resources. In addition to its process's context, a thread contains its own execution context which includes its runtime stack, the state of the machine's registers and several attributes (e.g., scheduling priority).⁵¹

When the system initializes a process, the process creates a **primary thread**. It acts as any other thread, except that if the primary thread returns, the process terminates, unless the primary thread explicitly directs the process not to terminate. A thread can create other threads belonging to its process.⁵² All threads belonging to the same process share that process's virtual address space. Threads can maintain their own private data in **thread local storage (TLS)**.

Fibers

Threads can create **fibers**, which are similar to threads, except that a fiber is scheduled for execution by the thread that creates it, rather than the scheduler. Fibers make it easier for developers to port applications that employ user-level threads. A fiber executes in the context of the thread that creates the fiber.⁵³

It fiber must maintain state information, such as the next instruction to execute processor registers. The thread stores this state information for each fiber. The thread itself is also a unit of execution, and thus must convert itself into a fiber to separate its own state information from other fibers executing in its context. In fact, the Windows API forces a thread to convert itself into a fiber before creating or scheduling other fibers. The thread's context remains, and all fibers associated with that thread execute in that context.⁵⁴

Whenever the kernel schedules a thread that has been converted to a fiber for execution, the converted fiber or another fiber belonging to that thread runs. Once a fiber obtains the processor, it executes until the thread in whose context it executes is preempted, or the fiber switches execution to another fiber within that thread. Just as threads possess their own thread local storage (TLS), fibers possess their own **fiber local storage (FLS)**, which functions for fibers exactly as TLS functions for a thread. A fiber can also access its thread's TLS. If a fiber deletes itself (i.e., terminates), its thread terminates.⁵⁵

Thread Pools

Windows XP also provides each process with a thread pool that consists of a number of worker threads, which are kernel-mode threads that execute functions specified by user threads. Because the functions specified by user threads may outnumber worker threads, Windows XP maintains requests to execute functions in a queue. The thread pool consists of worker threads that sleep until a request is queued to the pool.⁵⁶ The thread that queues the request must specify the function to execute and must provide context information.⁵⁷ The thread pool is created the first time a thread submits a function to the pool.

Thread pools have many purposes. Web servers and databases can use them to handle client requests (e.g., from Web browsers). Instead of incurring the costly overhead of creating and destroying a thread for each request, the process simply queues the request to its pool of worker threads. Also, several threads that spend most of their time sleeping (e.g., waiting for events to occur) can be replaced by a single worker thread that awakens each time one of these events occurs. Furthermore, applications can use the thread pool to accomplish asynchronous I/O by queuing a request to its pool of worker threads to execute the I/O completion routines. Using thread pools can make an application more efficient and simpler, because developers do not have to create and delete as many threads. However, thread pools transfer some control from the programmer to the system, which can introduce inefficiency. For example, the system grows and shrinks the size of a pro-

cess's thread pool in response to request volume; in some cases, the programmer can better estimate how many threads are needed.⁵⁸

Thread States

In Windows XP, threads can be in any one of eight states (Fig. 4.8). A thread begins in the **initialized state** during thread creation. Once initialization concludes, the thread enters the **ready state**. Threads in the ready state are waiting to use a processor. A thread that the dispatcher has decided will execute next enters the **standby state** as it awaits its turn for a processor. A thread is in the standby state, for example, during the context switch from the previously executing thread to that thread. Once the thread obtains a processor, it enters the **running state**. A thread transitions out of the **running state** if it terminates execution, exhausts its quantum, is preempted, is suspended or waits on an object. When a thread completes its instructions, it enters the **terminated state**. The system does not necessarily delete a **terminated** thread immediately, which can reduce thread creation overhead if the process reinitializes that thread. The system deletes a thread after its resources have been freed. If a **running** thread is preempted or exhausts its quantum, it returns to the **ready state**. If a **running** thread is preempted or exhausts its quantum, it returns to the **ready state**.

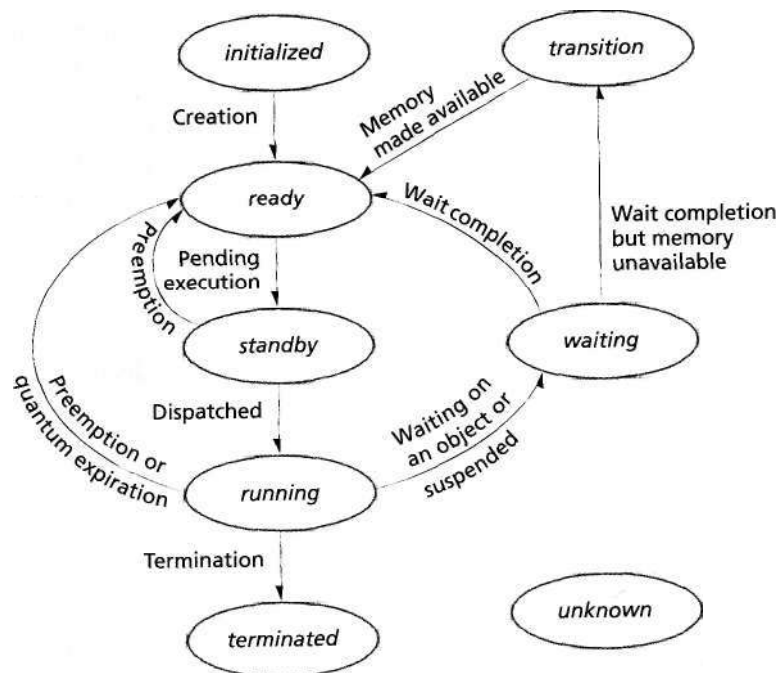


Figure 4.8 | Windows XP thread state-transition diagram.

A *running* thread enters the **waiting state** while waiting for an event (e.g., an I/O completion event). Also, another thread (with sufficient access rights) or the system can suspend a thread, forcing it into the *waiting* state until the thread is resumed. When the thread completes its wait, it either returns to the *ready* state or enters the **transition state**. The system places a thread in the *transition* state if the thread's data is not currently available (e.g., because the thread has not executed recently and the system needed its memory for other purposes), but the thread is otherwise ready to execute. The thread enters the *ready* state once the system pages the thread's kernel stack back into memory. The system places a thread in the **unknown state** when the thread's state is unclear (usually because of an error).^{59,60}

Self Review

1. What purpose do fibers serve in Windows XP?
2. How does a primary thread differ from other threads?

Ans: 1) Fibers exist to improve compatibility with applications that schedule their own threads (e.g., user-level threads). 2) Unlike other threads, when a primary thread returns, the process to which it belongs terminates.

4.11 Java Multithreading Case Study, Part I: Introduction to Java Threads

As we will see in subsequent chapters, the concurrency introduced by processes and threads has significantly impacted software design. In this and the following optional case studies, we provide examples of real programs that demonstrate and solve problems introduced by concurrency. We have chosen to implement these programs in Java due to its portability across most popular platforms. This case study assumes a basic knowledge of Java.

In this section, we overview various thread-related methods in the Java API. We use many of these methods in live-code examples. The reader should refer to the Java API directly for more details on using each method, especially the exceptions thrown by each method (see java.sun.com/j2se/1.4/docs/api/java/lang/Thread.html).

Class `Thread` (package `java.lang`) has several constructors. The constructor

```
public Thread( String threadName )
```

constructs a `Thread` object whose name is `threadName`. The constructor

```
public Thread()
```

constructs a `Thread` whose name is "Thread-" concatenated with a number, like `Thread-1`, `Thread-2`, and so on.

The code that "does the real work" of a thread is placed in its **`run`** method. The `run` method can be overridden in a subclass of `Thread` or it may be imple-

mented in a **Runnable** object; Runnable is a Java interface that allows the programmer to control a thread's life cycle with the run method in an object of a class that does not extend Thread.

A program launches a thread's execution by calling the thread's **start** method, which, in turn, calls method run. After start launches the thread, start returns to its caller immediately. The caller then executes concurrently with the launched thread.

The static method **sleep** is called with an argument specifying how long (in milliseconds) the currently executing thread should sleep; while a thread sleeps, it does not contend for the processor, so other threads can execute. This can give lower-priority threads a chance to run.

Method **setName** sets a thread's name, which facilitates debugging by allowing the programmer to identify which thread is executing. Method **getName** returns the name of the Thread. Method **toString** returns a String consisting of the name of the thread, the priority of the thread and the thread's **ThreadGroup** (analogous to the parent process of a group of threads). The static method **currentThread** returns a reference to the currently executing Thread.

Method **join** waits for the Thread to which the message is sent to terminate before the calling Thread can proceed; no argument or an argument of 0 milliseconds to method join indicates that the current Thread will wait forever for the target Thread to die before the Thread proceeds. Such waiting can be dangerous; it can lead to two particularly serious problems called deadlock and indefinite postponement—we discuss these concepts in Chapter 7, Deadlock and Indefinite Postponement.

Java implements thread signaling using the **interrupt** method. Calling the **interrupt** method on a thread that is blocked (because that thread's wait, join or sleep method was called) raises (throws) its InterruptedException exception handler. The static method **interrupted** returns true if the current thread has been interrupted and false otherwise. A program can invoke a specific thread's **isinterrupted** method to determine whether that thread has been interrupted.

Creating and Executing Threads

Figure 4.9 demonstrates basic threading techniques, such as constructing Thread objects and using Thread method sleep. The program creates three threads of execution. Each thread displays a message indicating that it is going to sleep for a random interval from 0 to 5000 milliseconds, then goes to sleep. When each thread awakens, it displays its name, indicates that it is done sleeping, terminates and enters the *dead* state. You will see that method main (i.e., the main thread of execution) terminates before the application terminates. The program consists of two classes—ThreadTester (lines 4-23), which creates the three threads, and PrintThread (lines 25-59), which contains in its run method the actions each PrintThread will perform.

Class PrintThread (lines 25-59) extends Thread, so that each PrintThread object can execute concurrently. The class consists of instance variable sleepTime

```

1 // Fig. 4.9: ThreadTester.Java
2 // Multiple threads printing at different intervals.
3
4 public class ThreadTester {
5
6     public static void main( String [] args )
7     {
8         // create and name each thread
9         PrintThread thread1 = new PrintThread( "thread1" );
10        PrintThread thread2 = new PrintThreadC "thread2" );
11        PrintThread thread3 = new PrintThreadC "thread3" );
12
13        System.err.println( "Starting threads" );
14
15        thread1.start0; // start thread1; place it in ready state
16        thread2.start(); // start thread2; place it in ready state
17        thread3.start(); // start thread3; place it in ready state
18
19        System.err.println( "Threads started, main ends\n" );
20
21    } // end main
22
23 } // end class ThreadTester
24
25 // class PrintThread controls thread execution
26 class PrintThread extends Thread {
27     private int sleepTime;
28
29     // assign name to thread by calling superclass constructor
30     public PrintThreadC String name )
31     {
32         super( name );
33
34         // pick random sleep time between 0 and 5 seconds
35         sleepTime = ( int ) ( Math. random() * 5001 );
36     } // end PrintThread constructor
37
38     // method run is the code to be executed by new thread
39     public void run()
40     {
41         // put thread to sleep for sleepTime amount of time
42         try {
43             System.err.println(    getName()    + " going to sleep for " +
44                                 sleepTime + " milliseconds" );
45

```

Figure 4.9 | Java threads being created, starting, sleeping and printing. (Part 1 of 2.)

```

46         Thread.sleep( sleepTime );
47     } // end try
48
49     // if thread interrupted during sleep, print stack trace
50     catch ( InterruptedException exception ) {
51         exception.printStackTrace();
52     } // end catch
53
54     // print thread name
55     System.err.println( getName0 + " done sleeping" );
56
57 } // end method run
58
59 } // end class PrintThread

```

Sample Output 1:

```

Starting threads
Threads started, main ends

```

```

thread1 going to sleep for 1217 milliseconds
thread2 going to sleep for 3989 milliseconds
thread3 going to sleep for 662 milliseconds
thread3 done sleeping
thread1 done sleeping
thread2 done sleeping

```

Sample Output 2:

```

Starting threads
thread1 going to sleep for 314 milliseconds
thread2 going to sleep for 1990 milliseconds
Threads started, main ends

```

```

thread3 going to sleep for 3016 milliseconds
thread1 done sleeping
thread2 done sleeping
thread3 done sleeping

```

Figure 4.9 | Java threads being created, starting, sleeping and printing. (Part 2 of 2.)

(line 27), a constructor (lines 29-36) and a run method (lines 38-57). Variable `sleepTime` stores a random integer value chosen when a new `PrintThread` object's constructor is called. Each thread controlled by a `PrintThread` object sleeps for the amount of time specified by the corresponding `PrintThread` object's `sleepTime`, then outputs its name.

The `PrintThread` constructor (lines 29-36) initializes `sleepTime` to a random integer from 0 to 5000. When a `PrintThread` is assigned a processor for the first time, its `run` method begins executing. Lines 43-44 display a message indicating the name of the currently executing thread and stating that the thread is going to sleep for a certain number of milliseconds. Line 43 uses the currently executing thread's `getName` method to obtain the thread's name, which was specified as a string argument to the `PrintThread` constructor and passed to the superclass `Thread` constructor in line 32. Note that line 43 uses `System.err` to print the message because `System.err` is unbuffered, meaning that it prints its argument immediately after being called. Line 46 invokes static `Thread` method `sleep` to place the thread into the *sleeping* state. At this point, the thread loses the processor, and the system allows another thread to execute. When the thread awakens, it reenters the *ready* state, where it waits until the system assigns it a processor. When the `PrintThread` object enters the *running* state again, line 55 outputs the thread's name in a message that indicates the thread is done sleeping; then method `run` terminates. This places the thread in the *dead* state, at which point its resources can be freed.

`ThreadTester` method `main` (lines 6-21) creates and names three `PrintThread` objects (lines 9-11). When debugging a multithreaded program, these names identify which threads are executing. Lines 15-17 invoke each thread's `start` method to transition all three `PrintThread` objects from the *born* state to the *ready* state. Method `start` returns immediately from each invocation; then line 19 outputs a message indicating that the threads were started. Note that all code in this program, except for the code in method `run`, executes in the main thread. The `PrintThread` constructor also executes in the main thread, since each `PrintThread` object is created in method `main`. When method `main` terminates (line 21), the program itself continues running, because there are still threads that are alive (i.e., the threads were started and have not yet reached the *dead* state) and are not daemon threads. The program terminates when its last thread dies. When the system assigns a processor to a `PrintThread` for the first time, the thread enters the *running* state, and the thread's `run` method begins executing.

The sample outputs for this program show each thread's name and sleep time as the thread goes to sleep. The thread with the shortest sleep time normally awakens first, indicates that it is done sleeping and terminates. Note in the second sample output that `thread1` and `thread2` each output their name and sleep time before method `main` completes its execution. This situation demonstrates that, once multiple threads are in the *ready* state, any thread can be assigned the processor.

Self Review

1. Why should programmers name Java threads?
2. How does Java allow programmers to create threads from objects that do not extend `Thread`?

Ans: 1) Naming Java threads allows the programmer to identify each thread when debugging. 2) A class can implement the `Runnable` interface so that objects of the class can control the life cycles of threads with their `run` methods.

Web Resources

www.serpentine.com/~bos/threads-faq/

Contains a frequently-asked-questions (FAQ) page about threads. Aside from answering basic thread questions, it provides links to books on threads, discusses some common pitfalls programmers encounter when implementing operating system support for threads and directs readers to thread packages available on the Internet.

www.linux-mag.com/2001-05/compile_01.html

This *Linux Magazine* article by Benjamin Chelf gives an overview of thread implementations. Chelf describes in detail how Linux handles Pthreads, providing sample Linux code.

publib.boulder.ibm.com/iserivs/v5r1/ic2924/index.htm?info/apis/rzah4mst.htm

Provides a complete API for programming Pthreads and defines basic thread concepts.

sources.redhat.com/pthreads-win32/

Contains an implementation of Pthreads for 32-bit Windows environments.

Java.sun.com/docs/books/tutorial/essential/threads/

Sun's tutorial on Java threads.

java.sun.com/j2se/1.4.1/docs/api/

Java API containing a specification of the Thread class.

archive.devx.com/dotnet/articles/pa061702/pa061702-l.asp

Article that overviews multithreading using the .NET platform, including a concise presentation of multithreading concepts.

www.microsoft.com/msj/0499/pooling/pooling.aspx

Article that describes thread pooling in Windows XP.

java.sun.com/docs/hotspot/threads/threads.html

White paper that documents the relationship between Java threads and Solaris threads.

developer.apple.com/documentation/MacOSX/Conceptual/SystemOverview/InverEnvironissues/chapter_14_section_3.html

Describes the thread manager in Macintosh OS X.

Summary

In recent years, several general-purpose programming languages, such as Java, C#, Visual C++ .NET, Visual Basic .NET and Python, have made concurrency primitives available to the applications programmer. The programmer specifies that applications contain threads of execution, each thread designating a portion of a program that may execute concurrently with other threads. This technology, called multithreading, gives the programmer powerful capabilities. Operating system support for threads is essential to supporting languages that provide multithreading semantics.

Threads, sometimes called lightweight processes (LWPs), share many resources—most notably the address space of their process—to improve the efficiency with which they perform their tasks. The name "thread" refers to a single thread of instructions or thread of control; threads within a process execute concurrently to attain a common goal. Resources such as processor registers, the stack and other thread-specific data (TSD) are local to each thread, while all threads share the address space of their process. Depending on the thread implementation, threads may be managed by the operating system or by a user application. Although many operating systems support threads, the implementations vary considerably.

Threads have become prominent due to trends in software design, scalability and cooperation. Each thread transitions among a series of discrete thread states. Threads

and processes have many operations in common, such as create, exit, resume and suspend. Unlike process creation, thread creation does not require the operating system to initialize resources that are shared between the parent process and its threads (e.g., the address space). This reduces the overhead of thread creation and termination when compared to process creation and termination. Some thread operations do not correspond precisely to process operations, such as *cancel* and *join*.

Thread implementations vary among operating systems, but almost all operating systems support user-level threads, kernel-level threads or a combination of the two. User-level threads perform threading operations in user space, meaning that threads are created by runtime libraries that cannot execute privileged instructions or access kernel primitives directly. User-level thread implementations are also called many-to-one thread mappings, because the operating system maps all threads in a multithreaded process to a single execution context. Although user-level threads vary in performance among applications, many of their deficiencies relate to the fact that the kernel views a multithreaded process as a single thread of control.

Kernel-level threads attempt to address the limitations of user-level threads by mapping each thread to its own execution context. As a result, kernel-level threads provide a one-to-one thread mapping. Their benefits

178 Thread Concepts

include increased scalability, interactivity and throughput. Due to overhead and reduced portability, however, kernel-level threads are not always the optimal solution for multi-threaded applications.

The combination of the user- and kernel-level thread implementation is known as the many-to-many thread mapping. Some refer to this technique as *m-to-n* thread mapping because the number of user and kernel threads need not be equal. Many-to-many thread mappings reduce overhead due to one-to-one thread mappings by implementing thread pooling, which allows an application to specify the number of kernel-level threads it requires. Persistent kernel threads that occupy the thread pool are called worker threads. A technique that enables a user-level library to schedule its threads is called a scheduler activation. It occurs when the operating system calls a user-level threading library that determines if any of its threads need rescheduling. Scheduler activations can reduce context-switching overhead due to inappropriate scheduling decisions in the kernel by allowing the threading library to determine the scheduling policy that best addresses the application's needs.

When the operating system delivers a signal to a process, the process pauses execution and invokes a signal handler to respond. There are two types of signals: synchronous (which occur as the direct result of an instruction being executed) or asynchronous (which occur as the result of an event unrelated to the instructions being executed). Signal delivery introduces several design challenges in threading implementations. The threading library must determine each signal's recipient so that asynchronous signals are delivered properly. Each thread is usually associated with a set of pending signals that are delivered when it executes. One way to control which threads receive signals of a particular type is to require that each thread specify a signal mask that disables signals of particular types. A thread can thereby mask all signals except those that it wishes to receive.

Thread termination, or cancellation, also differs between thread implementations. Because multiple threads share the same address space, prematurely terminating a thread can cause subtle errors in processes. Some thread

implementations allow a thread to determine when it can be terminated to prevent the process from entering an inconsistent state.

Threads that use the POSIX threading API are called Pthreads (sometimes referred to as POSIX threads or POSIX 1003.1c threads). POSIX states that the processor registers, the stack and the signal mask are maintained individually for each thread, and any other resource information must be globally accessible to all threads in the process. POSIX specifies how operating systems should deliver signals to Pthreads in addition to specifying several thread-cancellation modes.

Linux allocates the same type of process descriptor to processes and threads, both of which are called tasks. Linux uses the UNIX-based system call `fork` to spawn child tasks. To enable threading, Linux provides a modified version named `clone`, which accepts arguments that specify which resources to share with the child task. At the highest level of resource sharing, tasks created by `clone` correspond to threads discussed in Section 4.2, Definition of Thread.

In Windows XP, threads—not processes—are dispatched to a processor; threads execute a piece of the process's code in the process's context, using the process's resources. In addition to its process's context, a thread contains its own execution context, which includes its runtime stack, the state of the machine's registers and several attributes (e.g., scheduling priority). When the system initializes a process, the process creates a primary thread, which typically terminates the process upon completion. Windows XP threads can create fibers, which are similar to threads, except that a fiber is scheduled for execution by the thread that creates it, rather than the scheduler. Windows XP provides each process with a thread pool that consists of a number of worker threads, which are kernel-mode threads that execute functions specified by user threads.

The Java programming language allows the application programmer to create threads that are portable to many computing platforms. Java uses class `Thread` to create threads, which execute code specified in a `Runnable` object's `run` method. Java supports operations such as naming, starting and joining threads.

Key Terms

asynchronous cancellation (POSIX)—Cancellation mode in which a thread is terminated immediately upon receiving the cancellation signal.

asynchronous signal—Signal generated for reasons unrelated to the current instruction of the running thread.

blocked state—Thread state in which the thread is awaiting notification of some event, such as completion of an I/O operation, before it can again become *ready*.

born state—Thread state in which a new thread begins its life cycle.

- C-threads**—Threads supported natively in the Mach micro-kernel (on which Macintosh OS X is built).
- cancellation of a thread**—Thread operation that terminates the target thread. Three modes of cancellation are disabled, deferred and asynchronous cancellation.
- dead state**—Thread state entered after a thread completes its task or otherwise terminates.
- deferred cancellation (POSIX)**—Cancellation mode in which a thread is terminated only after explicitly checking that it has received a cancellation signal.
- disabled cancellation (POSIX)**—Cancellation mode in which a thread does not receive pending cancellation signals.
- fiber** (Windows XP)—Unit of execution in Windows XP created by a thread and scheduled by the thread. Fibers facilitate portability for applications that schedule their own threads.
- fiber local storage (FLS)** (Windows XP)—Area of a process's address space where a fiber can store data that only the fiber can access.
- heavyweight process (HWP)**—A traditional process, which may contain one or more threads. The process is "heavyweight" because it is allocated its own address space upon creation.
- initialized state** (Windows XP)—Thread state in which the thread is created by the operating system.
- join**—Thread operation that causes the calling thread to block until the thread it joins terminates. A primary thread often joins each of threads it creates so that its corresponding process does not exit until all of its threads have terminated.
- kernel-level thread**—Thread created by an operating system (also called kernel thread).
- lightweight process (LWP)**—A single thread of program instructions (also called a thread of execution or thread of control). Threads are "lightweight" because they share their address space with other threads in the same process.
- main thread of execution**—Thread created upon process creation (also called primary thread).
- many-to-many (m-to-n) thread mapping**—Threading model in which a set of user threads is assigned to a set of kernel threads so that applications can benefit both from kernel-level threads and user-level features such as scheduler activations. In practice the number of user threads is greater than or equal to the number of kernel threads in the system to minimize memory consumption.
- many-to-one thread mapping**—Threading model in which all user-level threads in a process are assigned to one kernel thread.
- mask a signal**—Prevent a signal from being delivered. Signal masking enables a multithreaded process to specify which of its threads will handle signals of a particular type.
- multithreading**—Technique that incorporates multiple threads of execution within a process to perform parallel activities, possibly simultaneously.
- notify**—Thread operation that transitions its target thread from the *waiting* to the *ready* state.
- one-to-one mapping**—Threading model in which each user-level thread is assigned to a kernel-level thread.
- pending signal**—Signal that has not been delivered to a thread because the thread is not *running* and/or because the thread has masked signals of that type.
- primary thread**—Thread created upon process creation (also called main thread of execution). When the primary thread returns, its process terminates.
- Pthread (POSIX 1003.1c thread)**—Thread that conforms to the POSIX 1003.1c standard.
- ready (or runnable) state**—Thread state from which a thread can transition to the *running* state and execute on a processor. In Windows XP, a *ready* thread transitions to the *standby* state, from which it transitions to the *running* state.
- running state**—Thread state in which a thread executes on a processor.
- scheduler activation**—Mechanism that allows a user-level library to schedule kernel threads.
- signal handler**—Code that is executed in response to a particular signal type.
- signal mask**—Data structure that specifies which signals are not delivered to a thread. Depending on the signal type and default action, masked signals are either queued or dropped.
- sleep interval**—Period of time (specified by the thread that is about to enter the *sleeping* state) during which a thread remains in the *sleeping* state.
- sleeping state**—Thread state in which a thread cannot execute until being returned to the *ready* state after the sleep interval expires.
- standby state** (Windows XP)—Thread state denoting a thread that has been selected for execution.
- synchronous signal**—Signal generated due to execution of the currently *running* thread's instructions.
- task**—Linux representation of an execution context (i.e., process or thread).
- terminated state** (Windows XP)—Thread state that denotes a thread has finished executing.

180 Thread Concepts

thread—Entity that describes an independently executable stream of program instructions (also called a thread of execution or thread of control). Threads facilitate parallel execution of concurrent activities within a process.

thread local storage (TLS)—Windows XP implementation of thread-specific data. A thread can share its address space with other threads of that process, but hide its private data in its TLS.

thread pooling—Threading technique that employs a number of kernel threads that exist for the duration of the process that creates them. This technique can improve performance by reducing the number of costly thread creation and termination operations.

thread state—Status of a thread (e.g., *running*, *ready*, *blocked*, and so on).

transition state (Windows XP)—Thread state denoting a thread that has completed a wait but is not yet ready to

run because its kernel stack has been paged out of memory.

unknown state (Windows XP)—Thread state denoting that the some error has occurred and the system does not know the state of the thread.

user-level threads—Threading model in which all threads in a process are assigned to one execution context.

wake—Thread operation that transitions its target from the *waiting* state to the *ready* state.

waiting state—Thread state from which a thread cannot execute until transitioning to the *ready* state via a wake or notify operation.

Win32 thread—Threads natively supported in the Microsoft 32-bit Windows line of operating systems.

worker thread—Kernel thread that is a member of a thread pool. Worker threads may be mapped to any user thread in the process that created its thread pool.

Exercises

4.1 Compare and contrast thread dispatching in kernel-level threads and in user-level threads.

4.2 Would an algorithm that performs several independent calculations concurrently (e.g., matrix multiplication) be more efficient if it used threads, or if it did not use threads? Why is this a hard question to answer?

4.3 Name two situations other than a Web or database server for which thread pooling may be useful. Another possibility is an online interactive multi-player game. Several such games can have hundreds of active players simultaneously, each of whom can be represented by a thread. In these environments, users frequently enter and exit the game, meaning that thread pools can provide an efficient way to allocate threads. In general, any environment in which threads are rapidly created and destroyed, but variance in the total number of threads in the system at any given time is small, can benefit from thread pools.

4.4 Why are scheduler activations less portable than either user-level or kernel-level threads?

4.5 In Section 4.7.1, Thread Signal Delivery, we explained how signals may specify their targets using thread identifiers or process identifiers. Suggest an alternative way to implement signals to solve the asynchronous signal delivery problem.

4.6 Section 4.7.1, Thread Signal Delivery, discusses how Solaris 7 employed a thread called the Asynchronous Lightweight Process (ASLWP) to monitor and deliver pending asynchronous signals. How can this solution be simplified if the operating system represents threads using one-to-one thread mappings?

4.7 Compare and contrast asynchronous, deferred and disabled cancellation in Pthreads.

4.8 How do the Linux fork and clone system calls differ? How are they alike?

4.9 According to the discussion in Section 4.10, Windows XP Threads, which thread mappings does Windows XP support?

Suggested Projects

4.10 Research how threads are implemented in Windows XP, OS X, and Linux. Compare and contrast the implementations. How do these implementations differ from POSIX Pthreads?

4.11 Prepare a research paper on Java's implementation of threads. What information does the Java Virtual Machine keep

track of for each thread? Does the implementation depend on the platform on which the JVM is running?

4.12 Research the Solaris compatible Thread Library. Prepare a research paper on how this library works. How does it handle the thread signal delivery and termination problems

that were mentioned in the text? Does it conform to the POSIX standard?

4.13 Research three types of systems that use thread pooling. What average number of threads does each of these systems use?

Recommended Reading

The concept of a lightweight process originated in Sun Microsystems's implementation of lightweight processes.⁶¹ Lightweight processes soon became synonymous with threads, due to the development of the POSIX Pthreads specification and the introduction of the Java programming language. As programmers increasingly used threads in applications, operating system designers developed several levels of support for threads. One of the more successful models is Anderson et al.'s scheduler activations.⁶² The paper also discusses many issues relevant to user- and kernel-level threads. Likewise, Benjamin provides a clear discussion of threading models.⁶³ The reader will also find the evolution of multithreading support in Sun Microsystems's

4.14 Research how threads are managed in multiprocessor systems and distributed systems.

Solaris operating system to be particularly sensitive to operating system design concerns regarding thread implementations.⁶⁴

In recent years, the level of support for Pthreads across different operating systems has increased significantly. Blelloch describes Pthread implementation concerns and presents a case study on Solaris threads.⁶⁵ Butenhof's *Programming with POSIX Threads* discusses the POSIX specification and demonstrates how to program with Pthreads.⁶⁶ For more information regarding threads and thread programming, see Klemm et al.'s *Programming with Threads*⁶⁷ and Birrell's *An Introduction to Programming with Threads*.⁶⁸ Ousterhout discusses how improper use of threads can reduce performance.⁶⁹

Works Cited

1. "Process and Thread Functions," *MSDN Library*. Microsoft Corporation. February 2003, <msdn.microsoft.com/library/default.asp?url=/library/en-us/dl1p/proc/base/process_and_thread_functions.asp>.
2. Manuel Pavdn Valderrama, "The Unofficial GNU Mach IPC Beginner's Guide," August 19, 2002, <www.nongnu.org/hurdextras/interp/gui de/mach ipc threads.html>.
3. IEEE Standards Press, 1996, 9945-1:1996 (ISO/IEC) [IEEE/ANSI Std 1003.1 1996 Edition] *Information Technology-Portable Operating System Interface (POSIX)-Part 1: System Application: Program Interface (API) [C Language]* (ANSI).
4. Daley, Robert C., and Jack B. Dennis, "Virtual Memory, Processes, and Sharing in Multics," *Proceedings of the ACM Symposium on Operating System Principles*, January 1967.
5. "Multithreading in the Solaris Operating Environment, A Technical White Paper," Sun Microsystems, 2002.
6. O'Sullivan, B., "Answers to Frequently Asked Questions for comp.programming.threads: Part 1 of 1," Rev. 1.10, Last modified September 3, 1997, <www.serpentine.com/~bos/threads-faq/>.
7. "Lesson: Threads: Doing Two or More Tasks at Once," The Java Tutorial, Sun Microsystems, Inc., <java.sun.com/docs/books/tutorial/essential/threads/index.html>.
8. Appleton, R., "Understanding a Context Switching Benchmark," Modified October 23, 2001, <cs.nmu.edu/~randy/Research/Papers/Scheduler/understanding.html>.
9. "pthread_cancel," Documentation Overview, Glossary, and Master Index, © Digital Equipment Corporation 1996, All Rights Reserved; Product Version: Digital UNIX Version 4.0 or higher, March 1996, <www.cs.arizona.edu/computer.help/policy/DIGITAL_unix/AA-Q2DPC-TKTI.html/thrd0115.html#pt_cancel_13>.
10. "2.1.4 Waiting for a Hired to Terminate," Documentation Overview, Glossary, and Master Index, Product Version: Digital UNIX Version 4.0 or higher, March 1996, <www.cs.arizona.edu/computer.help/policy/DIGITAL_unix/AA-Q2DPC-TKTI.html/thrd0021.html#join_thread_sec>.
11. "Comp.os.research: Frequently Answered Questions, The History of Threads," Modified August 13, 1996, <www.faqs.org/faqs/os-research/part1/section-10.html>.
12. Anderson, T. E.; B. N. Bershad; E. D. Lazowska; and H. M. Levy, "Scheduler Activations: Effective Kernel Support for User-Level Management of Parallelism," *ACM Transactions on Computer Systems*, Vol. 10, No. 1, February 1992, p. 54.
13. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
14. Anderson, T. E.; B. N. Bershad; E. D. Lazowska; and H. M. Levy, "Scheduler Activations: Effective Kernel Support for User-Level Management of Parallelism," *ACM Transactions on Computer Systems*, Vol. 10, No. 1, February 1992, p. 54.
15. D.G. Feitelson, "Job Scheduling in Multiprogrammed Parallel Systems," *IBM Research Report RC19790*, October 1994, 2d rev., August 1997, p. 121.
16. C. Benjamin, "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.

182 Thread Concepts

17. Feitelson, D. G., "Job Scheduling in Multiprogrammed Parallel Systems," IBM Research Report RC 19790, October 1994, 2d rev., August 1997, p. 121.
18. "Package java.nio.channels," Java 2 Platform SE v1.4.2, 2003 <java.sun.com/j2se/1.4.2/docs/api/java/nio/channels/package-summary.html#multiplx>.
19. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
20. Feitelson, D. G., "Job Scheduling in Multiprogrammed Parallel Systems," IBM Research Report RC 19790, October 1994, 2d rev., August 1997, p. 121.
21. Feitelson, D. G., "Job Scheduling in Multiprogrammed Parallel Systems," IBM Research Report RC 19790, October 1994, 2d rev., August 1997, p. 121.
22. Anderson, T. E.; B. N. Bershad; E. D. Lazowska; and H. M. Levy, "Scheduler Activations: Effective Kernel Support for User-Level Management of Parallelism," *ACM Transactions on Computer Systems*, Vol. 10, No. 1, February 1992, p. 54.
23. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
24. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
25. Richter, J., "New Windows 2000 Pooling Functions Greatly Simplify Thread Management," *Microsoft Systems Journal*, April 1999, <www.microsoft.com/msj/0499/pooling/pooling.aspx>.
26. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
27. Benjamin, C., "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
28. "Multithreading in the Solaris Operating Environment. A Technical White Paper," Sun Microsystems, 2002.
29. "I've Got Work To Do—Worker Threads and Work Queues," *AT Insider*, Vol. 5, No. 5, October 1998 (updated August 20, 2002), <www.osronline.com/article.cfm?id=65>.
30. "System Worker Threads," *MSDN Library*, June 6, 2003. <msdn.microsoft.com/en-us/kmarch/hh/kmarch/synchro_9ylz.asp>.
31. McCracken, D., "POSIX Threads and the Linux Kernel," *Proceedings of the Ottawa Linux Symposium 2002*, p. 335.
32. Troan, E., "A Look at the Signal API," *Linux Magazine*, January 2000, <www.linux-mag.com/2000-01/compile_01.html>.
33. Drepper, Ulrich, "Requirements of the POSIX Signal Model," modified May 17, 2003, <people.redhat.com/drepper/posix-signal-model.xml>.
34. "Multithreading in the Solaris Operating Environment, A Technical White Paper," Sun Microsystems, 2002.
35. "Multithreading in the Solaris Operating Environment, A Technical White Paper," Sun Microsystems, 2002.
36. Drepper, Ulrich, "Requirements of the POSIX Signal Model," modified May 17, 2003, <people.redhat.com/drepper/posix-signal-model.xml>.
37. Butenhof, David R., *Programming with POSIX Threads*, Boston, MA: Addison-Wesley, 1997, p. 144.
38. "POSIX Thread API Concepts," *IBM iSeries information Center*, <publib.boulder.ibm.com/iserics/v5r1/ic2924/index.htm?info/apis/whatare.htm>, modified October 16, 2002.
39. O'Sullivan, B., "Answers to Frequently Asked Questions for comp.programming.threads: Part 1 of 1," rev. 1.1, <www.serpentine.com/~bos/threads-faq/>, last modified September 3, 1997.
40. McCracken, D., "POSIX Threads and the Linux Kernel," *Proceedings of the Ottawa Linux Symposium 2002*, p. 332.
41. Butenhof, David R., *Programming with POSIX Threads*, Boston, MA: Addison-Wesley, 1997, pp. 214-215.
42. Drepper, Ulrich, "Requirements of the POSIX Signal Model," modified May 17, 2003, <people.redhat.com/drepper/posix-signal-model.xml>.
43. Butenhof, David R., *Programming with POSIX Threads*, Boston, MA: Addison-Wesley, 1997, pp. 143-144.
44. Blleloch, G. E., and G. J. Narlikar, "Pthreads for Dynamic and Irregular Parallelism," *Conference on High Performance Networking and Computing, Proceedings of the 1998 ACM/IEEE Conference on Supercomputing*, 1998, San Jose, CA.
45. Rangaraju, K., "Unravel the Complexity of Thread Programming," <www.fawcette.com/iavapro/2002_02/magazine/features/krangaraju/>.
46. "Threads Index," *The Open Group Base Specifications, Issue 6*, 2003, <www.opengroup.org/onlinepubs/007904975/idx/threads.html>.
47. Drepper, U., and I. Molnar, "The Native POSIX Thread Library for Linux," January 30, 2003, <people.redhat.com/drepper/nptl-design.pdf>.
48. "Multithreaded Programming Guide," Sun Microsystems, May 2002, pp. 11, 175.
49. Walton, S., "Linux Threads Frequently Asked Questions," January 21, 1997, <www.tldp.org/FAQ/Threads-FAQ/>.
50. Drepper, U., and I. Molnar, "The Native POSIX Thread Library for Linux," January 30, 2003, <people.redhat.com/drepper/nptl-design.pdf>.
51. "Processes and Threads," *MSDN Library*, <msdn.microsoft.com/library/en-us/dllproc/base/about_processes_and_threads.asp>.
52. "About Processes and Threads," *MSDN Library*, February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/about_processes_and_threads.asp>.
53. "Fibers," *MSDN Library*, February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/fibers.asp>.
54. "Convert Thread To Fiber," *MSDN Library*, February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/convertthreadtofiber.asp>.